Memory–Output Tradeoffs for Merging Dispersed Rotations in Downstream Quantum Compilers

Jinze Yang[1], Yangyang Li[2], and Xiu-Hao Deng[3,4]

[1]School of Physics, Xidian University, Xi'an 710071, China

[2]Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education of China, Collaborative Innovation Center of Quantum Information of Shaanxi Province, Institute of Interdisciplinary Quantum Science and Technology (IIQST), School of Artificial Intelligence, Xidian University, Xi'an 710071, China

[3]Shenzhen International Quantum Academy, Shenzhen 518048, China

[4]Shenzhen Branch, Hefei National Laboratory, Shenzhen, 518048, China

Time-dependent schedules split the angle of each rotation in a commuting layer across many rounds, and randomized compiling re-signs the pieces; when the per-round coefficients vary freely, no single instruction reveals the angle. We ask what a compiler that sees only the resulting instruction stream must pay to merge the pieces into one rotation per term. For a contiguous layer of m independent rotations on an accuracy-ϵ grid, we prove that any compiler that is correct on every such stream and reads it in forward passes must spend about $m \log_2(1/\epsilon)$ bits. They are counted as output committed while reading the earlier rounds plus state carried into the last round on each pass. A one-pass compiler that emits one aggregated rotation per term attains this to leading order, and so does a multipass compiler that merges a different block of terms on each pass. A one-pass stream optimizer with a much smaller buffer must emit unmerged output early. Memory and early output are not always interchangeable. Suppose k of the m terms, chosen at random, are split over two rounds, and the compiler emits all their first-round rotations before the second round, where they are applied. That prefix then needs on average at least $\log_2 \binom{m}{k}$ extra bits if each record must address its term, and a smaller amount, still linear in k for fixed $k/m < 1$, if it need only implement the rotations. A prefix buffered until the next round is read need not pay extra. Independently of the bound, on a four-qubit benchmark that repeats one diagonal layer, under one disclosed synthesis and surface-code model, Qiskit's highest preset optimization level, given the pre-lowered circuit, needs 385–1,753 times the logical T states of our compiler's output from the unlowered circuit, which merges the repetitions but not the decomposition of each controlled phase; the ratio grows with the number of repetitions. TKET's PauliSimp, also given a pre-lowered circuit, merges across that decomposition too and needs about half as many as ours.

Yangyang Li: yyli@xidian.edu.cn

Xiu-Hao Deng: dengxiuhao@iqasz.cn

1 Introduction

Two standard steps of a fault-tolerant software stack can split rotation angles. A time-dependent schedule of a commuting Hamiltonian, or a block of consecutive commuting layers with independent per-term parameters [1, 2, 3, 4, 5], applies an accumulated phase over several contiguous rounds, and when the per-round coefficients vary freely no single round reveals the total angle of a phase term. Randomized compiling then dresses each round with random Pauli gates, so that coherent errors average into stochastic Pauli noise [6, 7]; for a Z-type rotation, conjugation by a random Pauli flips the sign of its angle. Both steps are correct and useful, and together they leave the total angle spread over many re-signed instructions (figure 1a). Standard QAOA and VQE layers, and generic Trotter steps, are separated by noncommuting layers and do not by themselves form such blocks.

Whether the pieces are merged before gate synthesis matters for fault-tolerant cost. At accuracy ϵ a generic rotation costs about $3\log_2(1/\epsilon)$ T gates, roughly 30 at $\epsilon = 10^{-3}$ [8], every T gate consumes a distilled magic state [9, 10], and a term left in r pieces is synthesized r times. Merging commuting rotations is accordingly standard: phase folding and phase-polynomial methods [11, 12, 13, 14, 15], Pauli-gadget and ZX rewriting [16, 17, 18], and the Hamming-weight phasing of fault-tolerant resource estimates [19, 20] all exploit commutation. These methods assume an optimizer that sees, or can rebuild, a global description of the layer. Some compilation instead runs downstream of the steps that dispersed it: a transpiler service that receives a circuit already lowered to standard gates, for instance as OpenQASM [21]; a vendor pass that post-processes

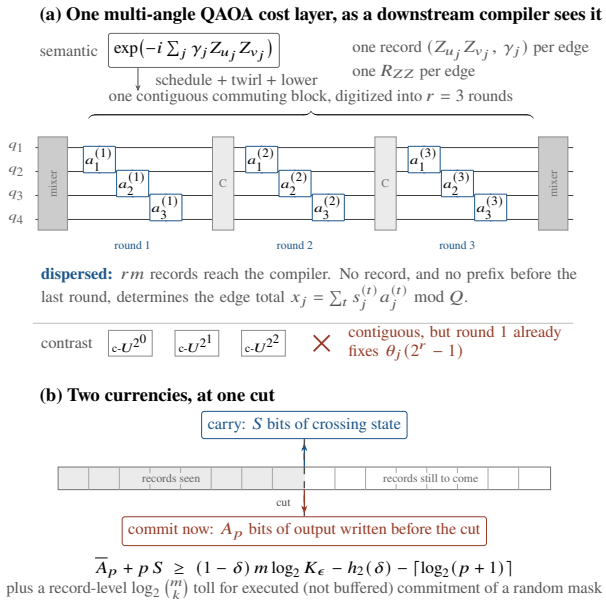


Figure 1: Dispersal and the two ways to undo it. (a) A diagonal cost layer digitized under a schedule that varies each grid-rounded edge coefficient freely across r contiguous rounds (a lowering we construct for a multi-angle QAOA cost layer) is optionally sign-randomized and reaches a downstream compiler as r rounds of grid residues; no record and no prefix that ends before the last round determines an edge total x_j. The bounds count $\mathbb{F}_2$-independent edge terms, $|V| - 1$ of them for a connected graph. A standard QAOA layer, in which every round rescales one weight vector, is not such a stream, and neither is a controlled-power ladder lowered exactly (bottom), whose first round fixes the total. Boxes marked C are Clifford records that permute the generators; the mixer layers bound the block. (b) At the boundary before the last round, the information about earlier rounds either crosses as compiler state or has already been committed as output. The $\log_2 \binom{m}{k}$ toll printed in (b) is the record-level one; commitment that need only implement the rotations pays a smaller toll, and buffered commitment pays none.

an emitted instruction stream; or the classical control electronics, which process the stream in real time and track Pauli gates in a frame [22]. We ask what a compiler in this position must pay to merge the pieces: how much information it must hold between rounds, and what it costs to commit output instead.

Our first result answers this question for a commuting layer of m independent Z-type rotations whose angles lie on an ϵ-calibrated grid and arrive split over r rounds (figure 1b). Any compiler that is correct on every such stream, makes at most p forward scans, and fails with probability at most $\delta < 1/2$ obeys

$$\overline{A}_p + pS \;\geq\; (1-\delta)\, m \log_2 K_\epsilon - h_2(\delta) - \lceil \log_2(p+1) \rceil \quad (1)$$

(theorem 2), where h_2 is the binary entropy function. Here $K_\epsilon = \Theta(1/\epsilon)$ is the number of angles of the grid, which is also the largest number of rotations about a common axis that are pairwise 4ϵ-separated; $K_\epsilon = 785$ at $\epsilon = 10^{-3}$. $\overline{A}_p$ is the expected number of output bits committed while the earlier rounds are scanned, summed over passes, and S caps the state carried across a crossing between the earlier rounds and the last; of the $2p - 1$ crossings that p forward scans make, only the p forward ones enter. S charges every channel through which a compiler restarted at a crossing could resume—RAM, caches, random seeds, any retained copy of the input, and any intermediate representation—so rebuilding a global phase table is allowed but paid for. The final output alone also needs $(1-\delta)m \log_2 K_\epsilon - h_2(\delta)$ bits. At $\delta = 0$, a one-pass compiler that keeps the running sums and emits one aggregated rotation per term attains equation (1) to leading order, and so does a p-pass compiler that merges a different block of terms on each pass, as long as p is small compared with $m/\log m$; a compiler given the layer in aggregated form needs only $O(\log m \cdot \log(1/\epsilon))$ bits of state (propositions 2 and 3).

Per term the price is $\log_2 K_\epsilon \approx 9.6$ bits at $\epsilon = 10^{-3}$. What equation (1) fixes is where those bits must live, and this is where dispersal enters. The inequality would hold even if the last round were known in advance, but output written before the last round arrives cannot yet contain the merged angles. A one-pass stream optimizer whose buffer is much smaller than $m \log_2 K_\epsilon$ bits must therefore commit about that much unmerged output early, and a deferred compiler, which commits nothing early, must carry all of it across the crossings (theorem 3). The proof is a standard multipass streaming argument [23, 24, 25], akin to streaming with a read-only input and a write-only output [26]; what is specific to compilation is the fooling family and the accounting. When the grid-rounded coefficients vary freely from round to round, time-dependent schedules and layerwise-parameterized blocks can emit exactly such fooling streams, with no added secret sharing. Randomized compiling keeps them fooling streams and does not create the dispersal; in frame-tracked form the pending Pauli frame is itself m bits that must cross or be committed, although a compiler that folds each sign into its stored residue pays nothing beyond equation (1) (propositions 4 and 5).

Equation (1) charges committed output and carried state at par. Our second result shows when they are not interchangeable. Take a two-round stream in which only k of the m terms are dispersed, the k drawn uniformly at random so that one compiler and one output code must serve every pattern, and suppose the compiler writes the first-round records of all k terms before the boundary and that this prefix is applied there, before the next round is read—as in a control stack that streams instructions to the device. If each record must address its term, the prefix needs on average at least $\log_2 \binom{m}{k}$ bits beyond its payload, about $h_2(k/m)/(k/m)$ bits per committed term, which a deferred compiler does not pay

(proposition 7). If the prefix need only implement the first-round rotations, a smaller toll remains, still linear in k: for fixed $\eta = k/m < 1$ its rate is $h_2(\eta(1-1/Q))/\eta - h_2(1/Q)$ bits per committed term, where Q is the number of grid angles: at $k = m/2$ this is 1.99 against 2 on the $\epsilon = 10^{-3}$ grid ($Q = K_\epsilon = 785$) and 0.92 against 2 on the coarse grid $Q = 3$ (corollary 2). A compiler that writes the same content irrevocably but applies it only after reading on pays nothing extra, so the toll requires the prefix to be executed at the cut; irrevocability alone does not create it. It resembles the advice costs of online algorithms [27, 28] and parallels relaxed-irrevocability models of online computation [29].

A software model of the two-round stream with $Q = 3$ and binary aggregates, which we call the masked-share device (each binary aggregate is hidden by its two shares until the second round), makes both results concrete; its state at the cut is serialized and checked by restarting from it. As the dispersed fraction of a layer varies, the state of a deferred compiler in this model grows within 5% of $\log_2 3 + 1$ bits per dispersed term at $m = 32$, 64, and 128: its serializer stores each residue and its frame bit separately, one bit above the proven floor of $\log_2 3$ (figure 2). Under a preregistered protocol, an encoder that ranks the dispersal pattern attains $\log_2 \binom{m}{k}/k$ to within 0.06 bits per term over a 16-fold range of m, and the buffered control pays zero (figure 3). These are constructive checks of a source-coding bound, not tests that could have failed, and two preregistered attempts to detect the toll in Qiskit and TKET output did not meet their design premise (section 5).

For external toolchains we report behavior. Tools that rebuild a global representation are not defeated by dispersal: on matched flat representations of a fixed-modulus target, including a masked modular-share stream, PyZX and TKET's PauliSimp (in its recommended native configuration) reach one rotation per active term. The bounds price that rebuild in state; they do not forbid it. We also propagate rotation counts through checked Clifford+T synthesis and two surface-code profiles under a common error budget (section 7). On a four-qubit benchmark that repeats one diagonal layer, Qiskit's optimization level 3, given the pre-lowered circuit, keeps about 12 rotations per repetition, while our reference compiler, a representation-aware pass in the Unitary Compiler Collection (UCC), given the unlowered circuit, keeps 22 in total. Across 48 settings at each of two sizes, Qiskit's output needs 385–1,753 times the logical T states of ours, a ratio that grows with the number of repetitions. Our reference merges the repetitions but keeps the three phase terms into which each controlled phase decomposes (a full merge keeps 10); TKET's PauliSimp, given the pre-lowered circuit, merges those too and needs 0.44–0.54 times our T states. The inputs differ between pipelines and the benchmark repeats a layer rather than dispersing it, so these numbers follow from one disclosed model, not from equation (1), which counts bits rather than gates.

The bounds concern compilers that see the stream after scheduling, randomization, and lowering, and that read it in forward scans. They need the rounds of a layer to be contiguous, up to Clifford records that permute the generators (lemma 1), and the coefficients to vary freely from round to round. Non-commuting interleavings, reverse scans, and uncharged random access to the input lie outside them, and a compiler that sees the layer before scheduling and lowering escapes them. Read the other way, these limits are a design rule: where the interface and latency budget allow, randomize and lower late, and keep phase semantics until the coefficients have been merged. We do not exhibit a production pipeline that emits such streams to a memory-bounded downstream stage.

2 Model

Throughout, $R_P(\theta) = \exp(-i\theta P/2)$, and Pauli-$Z$ characters $P_j = Z^{a_j}$, $a_j \in \mathbb{F}_2^n$, are *independent* if $\operatorname{rank}_{\mathbb{F}_2}(a_1, \ldots, a_m) = m$. Correctness is measured by the projective distance

$$d_{\mathrm{proj}}(U, V) = \min_{\phi \in \mathbb{R}} \|U - e^{i\phi} V\|_{\mathrm{op}}. \tag{2}$$

It is within a factor of two of the diamond distance of the corresponding unitary channels, $d_{\mathrm{proj}} \le \|\mathcal{U} - \mathcal{V}\|_\diamond \le 2 d_{\mathrm{proj}}$ (lemma 2), so every lower bound below holds with the same threshold when correctness is measured in the unnormalized diamond norm.

2.1 The accuracy grid

The lower bounds rest on how far apart the target unitaries of a layer are, so we first compute that separation and fix the grid. For an alphabet $[K]_0$ and a step $\Delta > 0$ put $\operatorname{dist}_{2\pi}(a, 0) := \min_{z \in \mathbb{Z}} |a - 2\pi z|$,

$$\gamma(K, \Delta) := \min_{1 \le d \le K-1} \operatorname{dist}_{2\pi}(d\Delta, 0),$$

$$U_x := \prod_{j=1}^{m} R_{P_j}(\Delta x_j),$$

for $x \in [K]_0^m$ and independent characters $P_1, \ldots, P_m$.

Theorem 1 (Exact worst-pair separation)**.** *For every $m \ge 1$,*

$$\min_{x \ne y} d_{\mathrm{proj}}(U_x, U_y) = 2 \sin\left(\frac{\gamma(K, \Delta)}{4}\right). \tag{3}$$

If $\Delta = 2\pi/Q$ and $2 \le K \le Q$, then $\gamma(K, \Delta) = \Delta$: the full residue alphabet $K = Q$ is allowed although the coefficient interval $[0, (K-1)\Delta]$ wraps around the circle.

Proof. Fix $x \neq y$ and j with $s_j := x_j - y_j \neq 0$. Independence makes the joint sign map $z \mapsto (\chi_{P_1}(z), \ldots, \chi_{P_m}(z))$ surjective onto $\{\pm 1\}^m$, so the diagonal unitary $U_x U_y^\dagger$ has two eigenvalues, from sign patterns that agree off j, at circular angular separation $\mathrm{dist}_{2\pi}(|s_j|\Delta, 0) \geq \gamma$. After any common phase rotation, one of two circle points at angular separation $g \in [0, \pi]$ lies at chordal distance at least $2\sin(g/4)$ from 1; taking the infimum over the global phase gives the lower bound. A pair that differs in one coordinate by a minimizing d has exactly two relative eigenvalues at separation γ, and centering the shorter arc attains the bound. For $\Delta = 2\pi/Q$ and $1 \leq d \leq Q-1$, $\mathrm{dist}_{2\pi}(d\Delta, 0) = (2\pi/Q)\min\{d, Q-d\} \geq 2\pi/Q$, with equality at $d = 1$. □

Corollary 1 (The ϵ-calibrated grid)**.** *For $0 < \epsilon \leq 1/8$ put*

$$Q_\epsilon := \left\lfloor \frac{\pi}{2\arcsin(2\epsilon)} \right\rfloor, \qquad K_\epsilon := Q_\epsilon, \qquad \Delta_\epsilon := \frac{2\pi}{Q_\epsilon}. \tag{4}$$

Then $1/(4\epsilon) \leq K_\epsilon \leq \pi/(4\epsilon)$; the K_ϵ^m unitaries $\{U_x : x \in \mathbb{Z}_{Q_\epsilon}^m\}$ are pairwise at projective distance at least $2\sin(\pi/(2Q_\epsilon)) \geq 4\epsilon$; and Q_ϵ equals the largest number of rotations $R_P(\theta)$ about one axis that are pairwise 4ϵ-separated, i.e., at projective distance at least 4ϵ. At $\epsilon = 10^{-3}$, $K_\epsilon = 785$.

The proof is in Appendix A. The grid is therefore not a design choice: its alphabet is the metric-entropy count of distinguishable Z-rotations at the working separation, and generic angles on it cost $\Theta(\log(1/\epsilon))$ T gates under Ross–Selinger synthesis [8, Alg. 9.9]. No angle-by-angle T-count equality is claimed; special grid points, such as Clifford angles, are cheaper.

2.2 Dispersed streams and the charged compiler

Definition 1 (r-round dispersed stream)**.** *Fix $\mathbb{F}_2$-independent Pauli-Z characters $P_1, \ldots, P_m$ on $n \geq m$ qubits, a modulus $Q \geq 3$, the step $\Delta = 2\pi/Q$, an aggregate alphabet $[K]_0 = \{0, \ldots, K-1\}$ with $2 \leq K \leq Q$, and $r \geq 2$ rounds. For $x \in [K]_0^m$, shares $a^{(1)}, \ldots, a^{(r)} \in \mathbb{Z}_Q^m$ are valid if*

$$\sum_{t=1}^{r} a_j^{(t)} = x_j \pmod{Q} \qquad (j \in [m]). \tag{5}$$

The flat input *is the round-major record sequence* $\mathsf{Update}(t, j, a_j^{(t)})$*, which implements $W = \prod_t \prod_j R_{P_j}(\Delta a_j^{(t)})$; the* semantic input *is the addressed aggregate vector x, which specifies $U_x = \prod_j R_{P_j}(\Delta x_j)$.*

Proposition 1 (Semantic equivalence and token privacy)**.** *Every valid flat stream implements U_x up to global phase. No single record determines x_j: for every fixed (t, j, a) and every $b \in [K]_0$ there is a valid stream with $a_j^{(t)} = a$ and $x_j = b$. Under the canonical sharing distribution, in which the first $r-1$ shares are independent and uniform and the last is fixed by* (5)*, every individual residue is uniform on $\mathbb{Z}_Q$ and independent of x_j.*

Proof. Write $\sum_t a_j^{(t)} = x_j + Qk_j$ with $k_j \in \mathbb{Z}$. The P_j commute and $Q\Delta = 2\pi$, so $\prod_t R_{P_j}(\Delta a_j^{(t)}) = (-1)^{k_j} R_{P_j}(\Delta x_j)$, and the signs multiply to a global phase. For privacy, choose any one of the other $r-1$ residues to enforce the desired sum. A sum that contains an independent uniform residue is uniform, which gives the distributional claim. □

A compiler $\mathcal{A}$ is *charged* as follows. It may use random-access RAM, stacks, hash tables, caches, graph or DAG intermediate representations, and a random tape, and it may choose its number of passes dynamically. Its final output is write-once and is a codeword of a fixed prefix-free code. At every execution cut c we serialize a complete restart snapshot in five disjoint, self-delimiting fields: control (pass and finite-control state, heads, emitted-prefix lengths, random tape and cursor), store (RAM, stacks, hash tables, caches, including computed integer data such as partial sums at their literal bit length), window (live input tokens), parameter (every live formal angle parameter), and IR (every other live intermediate object, including any retained copy of consumed input). The charged budget is the total literal length

$$B_{\mathrm{cut}}(c) = B_{\mathrm{control}} + B_{\mathrm{store}} + B_{\mathrm{window}} + B_{\mathrm{parameter}} + B_{\mathrm{IR}}. \tag{6}$$

Rebuilding a global phase table, ZX graph, or Pauli network is therefore allowed but paid for. Records and parameters are serialized by a fixed self-delimiting codec, so an address on n qubits costs $\Theta(\log n)$ bits; the lower bounds below hold for every self-delimiting serialization whose control field records the emitted-prefix length, and only the additive terms of the upper bounds depend on the codec. The codec and the field-by-field implementation are specified in the Supplemental Material [30].

Definition 2 (Forward-pass accounting)**.** *Split the round-major stream into Alice's prefix A (rounds $1, \ldots, r-1$) and Bob's suffix B (round r). A forward p-pass compiler scans AB from left to right at most $p = p(m)$ times; each new pass is re-delivered by the source from the start of the stream, as in standard multipass streaming. A run therefore makes at most p forward crossings $A \to B$ and at most $p-1$ rewind crossings $B \to A$. For a run on input I, let $A_p(I)$ be the number of output bits committed while the machine scans A, summed over passes, and let $\Sigma_p^{\rightarrow}(I) := \sum_{t \leq p} B_{\mathrm{cut}}(c_{A \to B, t})$ be the total snapshot length at the forward crossings. For randomized compilers, $\overline{A}_p := \max_I \mathbb{E}_\rho A_p(I, \rho)$, $\overline{\Sigma_p^{\rightarrow}} := \max_I \mathbb{E}_\rho \Sigma_p^{\rightarrow}(I, \rho)$, and the crossing cap is $S := \max_I \mathbb{E}_\rho[\max_c B_{\mathrm{cut}}(c; I, \rho)]$ over*

the $2p-1$ crossings, so that $\overline{\Sigma}_p^{\rightarrow} \le pS$. Unbarred symbols denote maxima over valid inputs.

Definition 3 (Output measures)**.** *The self-contained code E_{self} serializes a target-basis circuit with all opcodes, addresses, delimiters, and parameters. Given the public dictionary $\mathcal{F}_m = (m, P_1, \dots, P_m, K, \Delta)$, the family-relative code $E_{\mathrm{rel}}(\cdot \mid \mathcal{F}_m)$ may instead write the aggregate digits packed in base K (aggregate mode), complete target-basis records (literal mode), or, under a header that declares a public schedule, residues packed in base Q without addresses (ordered-share and p-block hybrid modes); the modes are specified in Appendix A. We write B_{rel} and B_{self} for the code lengths of an output, and B^{out} and $\overline{B}^{\mathrm{out}}$ for their worst-input value and worst-input expectation. Description bits, self-contained bits, and gate counts are bounded separately and never converted into one another.*

3 Lower bounds and matching constructions

We now bound what a charged compiler must spend on the streams of definition 1 and show that the bounds are attained. Put

$$L_\delta(m,K) := (1-\delta)m\log_2 K - h_2(\delta), \qquad 0 \le \delta < \tfrac{1}{2}. \tag{7}$$

Theorem 2 (K-ary randomized multipass recoverability frontier)**.** *Let $Q \ge 3$, $2 \le K \le Q$, and $0 \le \epsilon < \sin(\pi/(2Q))$. Let a possibly randomized compiler be correct within projective error ϵ, with probability at least $1-\delta$ for every $x \in [K]_0^m$ and every valid r-round dispersed representation of x. Assume that it makes at most $p = p(m)$ forward passes in the charged model of definition 2. Then:*

$$\overline{B}^{\mathrm{out}}_{\mathrm{rel}} \ge L_\delta(m,K), \qquad \overline{B}^{\mathrm{out}}_{\mathrm{self}} \ge L_\delta(m,K), \tag{8}$$

$$\overline{A}_p + \overline{\Sigma}_p^{\rightarrow} \ge L_\delta(m,K) - \lceil \log_2(p+1) \rceil. \tag{9}$$

In particular, for the worst-input expected crossing cap S of definition 2,

$$\boxed{\begin{aligned}&\overline{A}_p + pS \\ &\quad \ge (1-\delta)m\log_2 K - h_2(\delta) - \lceil \log_2(p+1) \rceil,\end{aligned}} \tag{10}$$

and a fortiori *$\overline{A}_p + (2p-1)S$ obeys the same bound. S includes every field of* (6)*; it is not a RAM-only bound. For the tuned choice of corollary 1, both right-hand sides are $\Omega((1-\delta)m\log(1/\epsilon))$ for fixed $\delta < 1/2$ and $p = 2^{o(m)}$.*

Proof. For (8), take X uniform on $[K]_0^m$ and fix one valid dispersed representation for each value of X. The packing separation is greater than 2ϵ, so nearest-packing decoding recovers X from a successful output. Fano's inequality gives

$$I(X;C) \ge m\log_2 K - h_2(\delta) - \delta\log_2(K^m - 1) \ge L_\delta(m,K).$$

Expected length in either separately fixed prefix-free output code is at least this mutual information; a worst-input expectation is at least the uniform average.

For (9), independently choose a uniform prefix aggregate $U \in \mathbb{Z}_Q^m$, put it in the first prefix round, put zero in any other prefix rounds, and set the last share $V = X - U \pmod Q$. Then V is uniform and independent of X, so $H(X \mid V) = m\log_2 K$. Run the compiler and record the forward transcript

$$T := \big(c,\ \mathsf{s}_1, O_1,\ \dots,\ \mathsf{s}_c, O_c,\ O_{\mathrm{fin}}\big),$$

where $c \in \{0, \dots, p\}$ is the number of forward crossings, written in $\lceil \log_2(p+1) \rceil$ bits; s_t is the five-field snapshot at the t-th crossing $A \to B$; O_t is the output written while scanning A just before that crossing; and O_{fin} is any output written in A after the last crossing (after a final rewind, or in a run that never reaches B). Given T and V, Bob reproduces the whole run. From s_t and V he simulates the scan of B, including the random tape and dynamic control that s_t serializes, and thereby obtains the Bob-side output and the rewind snapshot; the rewind snapshot is therefore a function of (s_t, V) and need not be transmitted. The control field of s_t records the emitted-prefix length, which gives $|O_t|$ once s_t is read, so s_t is placed before O_t. After the c-th crossing, O_{fin} completes a codeword of the prefix-free output code and is therefore self-delimiting. Hence T is a prefix-free code for each fixed V. It determines the final output, the output decoder recovers X with error at most δ, and conditional Fano yields

$$I(X;T \mid V) \ge L_\delta(m,K).$$

The expected length of a prefix-free code is at least this conditional information, and that length is $\lceil \log_2(p+1) \rceil + A_p + \Sigma_p^{\rightarrow}$. Maximizing over inputs proves (9). Since each of the at most p forward crossings has expected maximum budget at most S, the boxed form follows. The tuned consequence follows from corollary 1. □

Only the p forward crossings enter: of the $2p-1$ alternations that p forward scans make, as in the standard simulation of p passes by a $(2p-1)$-message protocol [23, 24], each rewind snapshot is a function of the preceding forward snapshot and of the last round. A random-access input oracle, uncharged rereading outside the counted passes, and reverse scans are different models. Neither (8) nor (10) converts self-contained bits into gates. For one pass and zero error the bound has an integer form, and a compact output forces the whole of it into the state:

Theorem 3 (One-pass zero-error bound)**.** *Let a deterministic one-pass compiler be correct to $\epsilon < \sin(\pi/(2Q))$ on every valid stream of definition 1 with $K \in \{Q-1, Q\}$. Then*

$$A_1 + B_{\mathrm{cut}}(c_{A\to B,1}) \ge \lceil m\log_2 Q \rceil. \tag{11}$$

If, moreover, the compiler writes its output in the family-relative code E_{rel} and every final output satisfies $B_{\mathrm{rel}} \le m \log_2 K + \kappa \log_2(m+2)$ for a constant κ, then

$$B_{\mathrm{cut}}(c_{A\to B,1}) \ge \lceil m \log_2 Q \rceil - \kappa \log_2(m+2). \quad (12)$$

Proof. For each $u \in \mathbb{Z}_Q^m$, take the canonical prefix with aggregate u (u in the first round, zero in the other prefix rounds). For $u \ne u'$ choose one common final share v: $v = 0$ if $K = Q$; if $K = Q-1$, at each coordinate at most two values of v_j make $u_j + v_j$ or $u'_j + v_j$ equal to the excluded aggregate $Q-1$, and $Q \ge 3$ leaves an allowed one. Both completions are valid and their aggregates differ wherever u and u' differ, so by theorem 1 no circuit is within ϵ of both targets. The suffix computation is a function of the snapshot and the suffix, and a run that halted inside the prefix would give one output for two different targets, so every run reaches the crossing and the map $u \mapsto \mathsf{s}_u \cdot O_A(u)$, the snapshot followed by the committed prefix, is injective. The snapshot is self-delimiting and records the length of the committed prefix, so these messages form a prefix-free set, and Kraft's inequality gives (11). For (12), fix any valid prefix, with aggregate u. The K^m final shares v with $u + v \in [K]_0^m$ reach every aggregate, so they yield K^m distinct codewords of E_{rel}, all extending the committed prefix and of length at most $m \log_2 K + \kappa \log_2(m+2)$. Kraft's inequality applied to their suffixes bounds every committed prefix by $\kappa \log_2(m+2)$, and the injectivity argument above, applied to the canonical prefixes, then bounds the snapshot. □

For $Q = 3$ and binary aggregates, theorem 3 is the two-round masked-share device executed in sections 4 and 5: a deferred compiler with compact output carries at least $\lceil m \log_2 3 \rceil - O(\log m)$ bits across the cut, whereas a compiler given the aggregate vector copies its bits into aggregate mode, where base-2 blocks are exact, and so emits $m + O(\log m)$ output bits with $O(\log m)$ state. The bounds are attained to leading order by explicit compilers, whose proofs are in Appendix A.

Proposition 2 (Block hybrid)**.** *Fix $p, h \ge 1$ and let $q = \min\{m, ph\}$. There is a deterministic zero-error p-pass compiler for definition 1 with $K = Q$ that aggregates q coordinates and passes the other update records through in the public p-block hybrid mode of E_{rel}. It obeys*

$$S \le \left\lceil \frac{q}{p} \right\rceil \log_2 Q + O(\log(mQrp)), \quad (13)$$

$$A_p \le (r-1)(m-q)\log_2 Q + O(r \log(mQrp)), \quad (14)$$

$$B_{\mathrm{rel}}^{\mathrm{out}} \le q \log_2 Q + r(m-q)\log_2 Q + O((p+r)\log(mQrp)). \quad (15)$$

In the compact-output corner $h = \lceil m/p \rceil$, this becomes

$$B_{\mathrm{rel}}^{\mathrm{out}} \le m \log_2 Q + O(p \log(mQrp)), \quad (16)$$

$$S \le \left\lceil \frac{m}{p} \right\rceil \log_2 Q + O(\log(mQrp)). \quad (17)$$

In the compact-output corner $q = m$ for any number of rounds, and for every q when $r = 2$, the construction satisfies $A_p + pS \le m \log_2 Q + O((p+r)\log(mQrp))$, and the same bound holds for $A_p + \Sigma_p^{\rightarrow}$. Compared with (9) and (10) at $\delta = 0$, whose right-hand side is $m \log_2 Q - \lceil \log_2(p+1) \rceil$, it is therefore optimal to leading order whenever $p \log(mQrp) = o(m \log Q)$; the additive terms are not claimed to be optimal, and for p comparable to m they dominate.

Proposition 3 (Semantic and randomized upper corners)**.** *Under the same public dictionary, fix a block length $b \ge 1$. A one-pass semantic compiler receiving x emits the aggregate-mode code with b-digit blocks and satisfies*

$$\begin{aligned} B_{\mathrm{rel}}^{\mathrm{out}} &\le m \log_2 K + \lceil m/b \rceil + O(\log(mQ)), \\ S &\le b \log_2 K + O(\log(mQ)), \end{aligned} \quad (18)$$

and implements the same U_x as the flat stream. With $b = \lceil \log_2(m+1) \rceil$ the state is $O(\log m \cdot \log Q)$ and the output is $m \log_2 K\,(1 + o(1))$. Blocking is needed: when $K = 2^e K_{\mathrm{odd}}$ with $K_{\mathrm{odd}} > 1$, a write-once streaming writer of the single integer $\sum_j x_j K^{j-1}$ in binary, least significant bit first, reading $x_1, x_2, \ldots$ in order, must hold at least $\lfloor m/2 \rfloor \log_2 K_{\mathrm{odd}} - 1$ bits after half the digits. For any $0 \le \delta < 1/2$, an input-independent abort coin that runs this exact compiler with probability $1 - \delta$ and otherwise emits a fixed short circuit has worst-input success at least $1 - \delta$ and expected relative output

$$(1-\delta)\big(m \log_2 K + \lceil m/b \rceil\big) + O(\log(mQ) + 1). \quad (19)$$

Thus the randomized family-relative output lower bound is matched to leading order, up to framing and the binary-entropy term. The explicit self-contained circuit, with at most m rotations, takes $O(m(\log m + \log Q))$ bits for constant-weight characters on $n = \mathrm{poly}(m)$ qubits; no matching self-contained-bit or gate-count claim is made.

4 Physical sources of dispersed streams

The streams of definition 1 are stated adversarially. Here three standard mechanisms supply them: time-dependent scheduling gives the additive decomposition, randomized compiling gives sign symmetry, and the synthesis grid of corollary 1 gives the modulus. A *scheduled stream* is the stream of definition 1 with $K = Q$ and signed records $\mathsf{Update}(t, j, a_j^{(t)}, s_j^{(t)})$, $s_j^{(t)} \in \{\pm 1\}$, implementing $\prod_t \prod_j R_{P_j}(\Delta s_j^{(t)} a_j^{(t)})$, with aggregate

$$x_j = \sum_{t=1}^{r} s_j^{(t)} a_j^{(t)} \bmod Q. \quad (20)$$

The residue $a_j^{(t)}$ is a round-t physical coefficient rounded to the grid and $s_j^{(t)}$ is a twirl variable ($s \equiv +1$ without twirling). In *folded* mode each sign is absorbed into the residue at emission, as in compile-time randomized compiling [6, 7]; in *tracked* mode the signs accumulate in a classical Pauli-frame register and are settled downstream [31, 22].

Lemma 1 (Clifford interleaving)**.** *Let the rounds of a scheduled stream be separated by Clifford records that are fixed across the family, and let D_{t-1} be the accumulated Clifford emitted before round t. If each D_{t-1} maps the generating set to itself up to sign and permutation, $D_{t-1}^\dagger P_j D_{t-1} = \varepsilon_j^{(t)} P_{\pi_t(j)}$, then moving every Clifford record to the end yields a scheduled stream on the same generators, followed by a fixed Clifford suffix, in which the round-t record of generator j contributes residue $a_j^{(t)}$ with sign $\varepsilon_j^{(t)} s_j^{(t)}$ to coordinate $\pi_t(j)$. The normalization is computable from the prefix, and every statement about scheduled streams applies to the normalized image.*

The hypothesis holds for Pauli-frame layers, hence for the twirls of randomized compiling, and for routing layers whose qubit relabeling permutes the generators, such as the reflection of a chain. It fails for a Clifford that maps some P_j to a product of generators, and it does not cover the non-Clifford R_X layers that separate the diagonal rounds of a generic Trotter step.

Proposition 4 (Physical instantiation)**.** *Each of the following mechanisms can emit scheduled streams. Assume that the grid-rounded coefficients may vary independently across the r rounds. Then every aggregate $x \in \mathbb{Z}_Q^m$ is realized by some valid stream, and no record, and no prefix that ends before the last round, determines it.*

(a) Time-dependent Trotterization *of a commuting $H(t) = \sum_j c_j(t) P_j$ with step τ: round t applies $R_{P_j}(2c_j(t_t)\tau)$, and $s \equiv +1$.*

(b) Layerwise-parameterized phase layers*: a depth-r block of commuting layers with free coefficients $\theta_j^{(t)}$.*

(c) Randomized compiling *of (a) or (b), in folded or tracked mode. With independent uniform twirl bits and the canonical sharing of proposition 1, every signed residue is uniform on $\mathbb{Z}_Q$ and independent of x_j.*

A constant schedule $a_j^{(t)} \equiv a_j$ is degenerate: its first round determines the aggregate.

The coefficient-freedom hypothesis carries the weight of proposition 4: the proposition records which streams can occur, not that a given pipeline produces them. Its two halves play different roles. The frontier needs only the earlier rounds to range over $\mathbb{Z}_Q^m$ and would hold with a fixed last round. The reading that early output must be unmerged needs the last round to be free as well; if the last-round residue of each term is confined to a few grid steps, as for a schedule with small increments, a compiler can commit part of each merged angle early. The rounds must also be contiguous up to the Clifford records of lemma 1. Contiguity arises structurally in controlled-power ladders and phase estimation, whereas coefficient freedom comes from time-dependent schedules and from layerwise-parameterized blocks such as multi-angle QAOA [2]. A standard QAOA cost layer, in which every round rescales one weight vector, is contiguous but not coefficient-free, so the bounds do not constrain it. For the edge characters $Z_u Z_v$ of a connected graph the rank is $|V| - 1$, and the bounds apply with $m = |V| - 1$ to a spanning tree. Finally, the bounds constrain compilers downstream of the schedule, the twirl, and the lowering; a compiler that holds the λ-bit seed of a pseudorandom twirl faces only 2^λ sign patterns, and the frame bound below falls from m to λ bits. The workloads of section 8 repeat identical terms within each commuting block or separate blocks by non-commuting layers; they exercise aggregation, not this bound.

Proposition 5 (Frame as cross-cut state)**.** *Let a compiler be correct within $\epsilon < \sin(\pi/(2Q))$ on every twirled scheduled stream.*

(i) In either mode, sign normalization maps the stream onto a valid stream of definition 1, so theorems 2 and 3 apply to the physical stream under their own hypotheses; with the tuned modulus Q_ϵ, $0 < \epsilon \le 1/8$, theorem 2 gives equation (1)*.*

(ii) The pending frame classes alone force the following. A deterministic, zero-error, one-pass compiler whose output committed before an inter-round cut c has length A_{pre} satisfies, in the worst case, $A_{\mathrm{pre}} + B_{\mathrm{cut}}(c) \ge m$. A randomized compiler correct with probability at least $1 - \delta$, $\delta < 1/2$, making at most p forward passes, has

$$\overline{A}_p + \overrightarrow{\overline{\Sigma}_p} \ge (1-\delta)m - h_2(\delta) - \lceil \log_2(p+1) \rceil. \quad (21)$$

The frame register is live classical restart state and is charged in the snapshot. The frame does not add to equation (1), because the target depends on the signs only through equation (20) and a deferred compiler, one whose committed prefix is only a header, may fold each pending sign into the residue it stores.

Real layers need not be dispersed in every term. For a coupled family, let each generator independently with probability η arrive in dispersed twirled form and otherwise in aggregate-exposed form, as a single round-two record that carries its aggregate. We call η the *dispersal fraction*; it is not the twirl strength, and the bound below does not use the twirl.

Proposition 6 (Dispersal–memory bound)**.** *Let Q, K, ϵ, δ, and p be as in theorem 2, and let the compiler be*

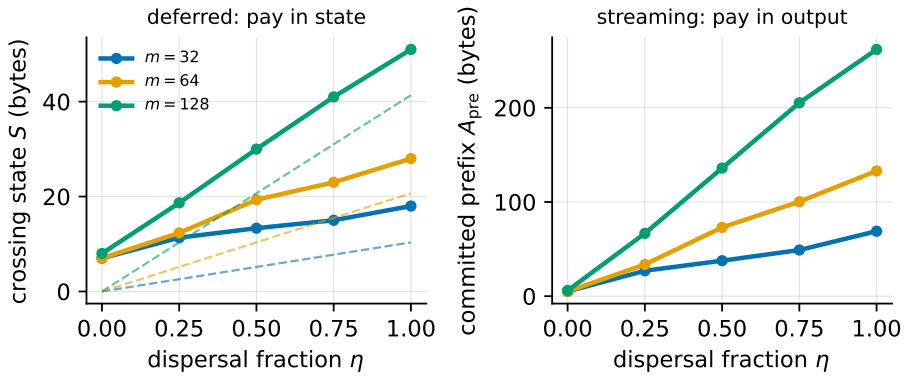


Figure 2: Measured dispersal–memory tradeoff on the executable two-round masked-share device ($Q = 3$, binary aggregates, tracked signs; every point restart-verified: the serialized snapshot plus the round-two suffix reproduces the exact output). Left: a deferred compact-output compiler pays the dispersal in crossing state; the measured slopes, $10.3/21.1/43.3$ bytes per unit η at $m = 32/64/128$, match its serializer's storage of one trit and one sign bit per dispersed generator, $\eta m(\log_2 3 + 1)/8$ bytes (dashed; ratios 0.99–1.05). The proven zero-error floor is $\eta m \log_2 3/8$ bytes (proposition 6). Right: a streaming compiler pays the same dispersal in committed prefix output (slopes $60/129/260$ bytes per unit η, larger by the public addresses) while its crossing state stays at ≤ 8 bytes. Frozen artifact: `data/frozen/eta_sweep_pareto.json`.

correct within ϵ with probability at least $1 - \delta$ on every valid stream for every mask. With the dispersal mask T drawn as above and $\overline{A}_p(T)$, $S(T)$ the worst-input quantities over streams with mask T,

$$\mathbb{E}_T\left[\overline{A}_p(T) + pS(T)\right] \geq (1-\delta)\,\eta m \log_2 K - h_2(\delta) - \lceil \log_2(p+1) \rceil. \tag{22}$$

For a deterministic one-pass zero-error compiler and $K \in \{Q-1, Q\}$, $\mathbb{E}_T[A_1(T) + B_{\mathrm{cut}}(T)] \geq \eta m \log_2 Q$.

Figure 2 executes equation (22) on the masked-share device of section 1, our executable implementation of the coupled family above with $Q = 3$, $K = 2$, $r = 2$, and tracked signs. The two payment channels of the bound appear as the two panels, and the exchange between them is linear in η with slope proportional to m. The panels also show an asymmetry that the frontier does not predict: per dispersed generator the state channel costs about 2.6 bits and the committed channel about 16, a ratio near 6:1 between two currencies that the frontier charges at par. The next section isolates the part of that gap that is not an artifact of the serializer.

5 Executed commitment

Isolating that part needs one hypothesis beyond the charged model, and the hypothesis is substantive: without it the gap is zero.

Definition 4 (Executed and buffered commitment)**.** *Consider a two-round stream with dispersal mask $T \subseteq [m]$: each coordinate in T has one addressed signed record in each round (kept even when its residue is zero), and each coordinate outside T has one aggregate-exposed record, of a distinguishable type, in round two. Suppose the compiler has irrevocably written the round-one content $W := (a_j^{(1)}, s_j^{(1)})_{j \in T}$ before the inter-round cut, as the prefix O_{pre} of length $A_{\mathrm{pre}} := |O_{\mathrm{pre}}|$ (A_1 of definition 2 for one pass). The commitment is* executed *if the prefix is applied at the cut, before any round-two record is parsed. It is executed* at record level *if it dispatches one addressed record per dispersed generator, as a control stack that forwards records to the device does, so that it determines (T, W) on its own; this is a modelling assumption, not a consequence of applying a unitary. It is executed* at unitary level *if it is only required, on its own and for every seed, to implement within ϵ the round-one action $\prod_{j \in T} R_{P_j}(\Delta s_j^{(1)} a_j^{(1)})$, which fixes $y \in \mathbb{Z}_Q^m$ with $y_j = s_j^{(1)} a_j^{(1)}$ on T and $y_j = 0$ off T, but neither T nor W. The commitment is* buffered *if the prefix is written once but applied only after later records have been read. A compiler is* mask-uniform *if one algorithm is correct on every mask, its committed prefixes over all masks form one prefix-free set, and the action of a prefix does not depend on T.*

Proposition 7 (Commitment toll)**.** *Fix $Q \geq 3$, $\epsilon < \sin(\pi/(2Q))$, and $1 \leq k \leq m$. Let $\mathcal{A}$ be mask-uniform and correct within ϵ on every stream of definition 4. Draw T uniformly among the $\binom{m}{k}$ subsets of size k and W uniformly on $\mathbb{Z}_Q^k \times \{\pm 1\}^k$, independently.*
(i) If $\mathcal{A}$ commits in record-level executed form, then

$$\mathbb{E}_{T,W}\left[A_{\mathrm{pre}}\right] \geq \underbrace{k \log_2(2Q)}_{payload} + \underbrace{\log_2 \tbinom{m}{k}}_{toll}, \tag{23}$$

whereas a deferred compiler attains $A_{\mathrm{pre}} = 0$ with crossing state $S \leq k \log_2(2Q) + O(\log m)$.

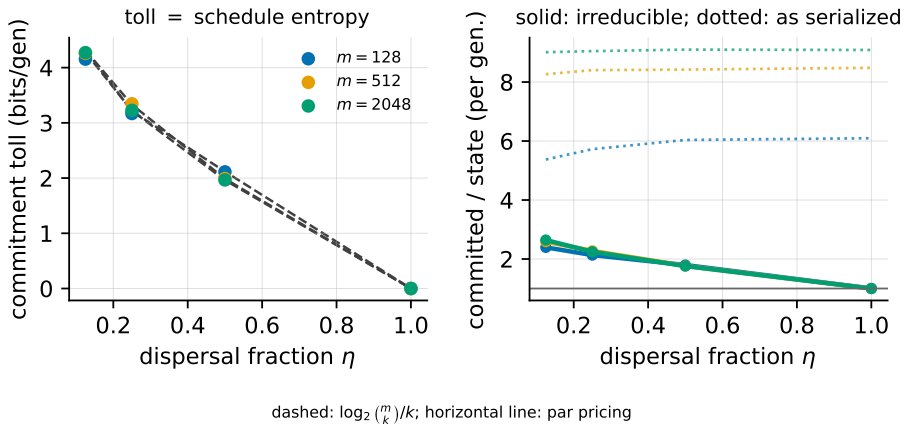


Figure 3: The record-level commitment toll attained on the masked-share device under a preregistered protocol; every cell restart-verified. This is a constructive check: the executed encoder attains the source-coding bound by construction. Left: bits per committed generator paid by an executed prefix in excess of the deferred snapshot, against $\log_2 \binom{m}{k}/k$ (dashed); the toll is flat in m and vanishes at $\eta = 1$, and the buffered control, whose prefix is the deferred snapshot, pays zero at every η. Right: the per-generator exchange rate between committed and state channels, under the ranking encoder (solid; at most 2.64, and 1 when the mask is public) and as serialized by the address-per-record codec of figure 2 (dotted), which is the source of the 6:1 ratio. Frozen artifact: `PaperDraft/generated/e8_commit_toll.json`.

Among prefixes that determine (T, W), ranking T among the $\binom{m}{k}$ masks attains equation (23) up to $O(\log m)$.

(ii) For $k = \Theta(m)$ with $k \le m - \Omega(m)$, the hypothesis in (i) cannot be removed: a compiler that writes W irrevocably in positional order and recovers T from the round-two records is admissible, commits in buffered form, and has $\mathbb{E}[A_{\mathrm{pre}}] \le k \log_2(2Q) + O(\log m)$.

Proof. (i) By definition 4 a record-level executed prefix determines (T, W), so $H(O_{\mathrm{pre}}) \ge H(T, W)$. By mask uniformity the committed prefixes form one prefix-free code over the joint source (T, W), and the expected length of a prefix-free code is at least the entropy of its source; hence $\mathbb{E}[A_{\mathrm{pre}}] = \mathbb{E}|O_{\mathrm{pre}}| \ge H(T, W)$. By construction, T and W are independent and uniform, so $H(T, W) = \log_2 \binom{m}{k} + k \log_2(2Q)$, which is (23), and a maximum dominates a mean. For the deferred compiler the round-major order is public, so its snapshot holds the k residues and signs in positional order while the round-two record types distinguish dispersed updates from aggregate-exposed coordinates and thereby disclose the mask at no charge; W is held as one packed integer of $\lceil k \log_2(2Q) \rceil$ bits in the store field, and the $O(\log m)$ term is the header. Ranking T among the $\binom{m}{k}$ masks gives a prefix of length $\lceil \log_2 \binom{m}{k} \rceil + \lceil k \log_2(2Q) \rceil + O(\log m)$ that determines (T, W), which shows the bound is tight up to the header in that class.

(ii) The mask-deferred compiler never rewrites an emitted record, so it meets the write-once rule of section 2.2, and it is correct because the round-two records supply T before the final output is written. Its prefix holds W in positional order at $k \log_2(2Q)$ bits plus a header, and reveals no function of T beyond the value of k already implied by its length, so no $\log_2 \binom{m}{k}$ term arises. There is no conflict with (i): this prefix does not determine (T, W), hence is not executable at the cut, and the compiler has bought its exemption by giving up the ability to apply what it has written. □

Corollary 2 (The toll at unitary level)**.** *Under the hypotheses of proposition 7, let $\mathcal{A}$ commit in unitary-level executed form, and let $\nu \sim \mathrm{Bin}(k, 1 - 1/Q)$ count the dispersed coordinates with nonzero signed residue. Then*

$$\mathbb{E}_{T,W}\left[A_{\mathrm{pre}}\right] \ge H(y) = \log_2 \binom{m}{k} + k \log_2 Q - \mathbb{E}\left[\log_2 \binom{m-\nu}{k-\nu}\right], \tag{24}$$

while a deferred compiler that folds each sign into its stored residue attains $A_{\mathrm{pre}} = 0$ with $S \le k \log_2 Q + O(\log m)$. For fixed $\eta = k/m$ and $m \to \infty$, the toll per committed generator tends to

$$\frac{h_2\big(\eta(1 - 1/Q)\big)}{\eta} - h_2(1/Q), \tag{25}$$

and this rate is attained by an executed prefix that writes ν, the rank of $\operatorname{supp} y$, and the nonzero values of y packed in base $Q - 1$. The rate is positive for $0 < \eta < 1$, vanishes at $\eta = 1$, and increases with Q to the record-level rate $h_2(\eta)/\eta$; at $\eta = \frac{1}{2}$ it is 0.918 bits for $Q = 3$ and 1.99 bits for $Q = 785$, against 2 at record level.

The proof of corollary 2 is in Appendix A. The toll is a property of executed commitment, not of commitment as such: a deferred or buffered compiler learns the mask from the round-two records. Where it is

paid, the record-level toll is at most $h_2(\eta)/\eta$ bits per generator, about 2 at $\eta = \frac{1}{2}$ and 0 at $\eta = 1$, and does not grow with m. For a single mask known in advance there is nothing to pay; mask uniformity is what makes the dispersal pattern a source. The two degeneracies of the unitary—a zero signed residue acts as the identity, and (s, a) and $(-s, -a)$ give the same rotation—are why the unitary-level toll is smaller. It is proved but not measured.

Figure 3 instantiates both clauses of proposition 7 on the masked-share device under a protocol frozen before the run (`PaperDraft/research/E8_preregistration.md`). It compares the deferred snapshot, the address-per-record serializer of figure 2, and an executed prefix that ranks the mask, at $m \in \{128, 512, 2048\}$ and up to $m = 32768$ for two encoders. The measured excess matches $\log_2 \binom{m}{k}/k$ within 0.06 bits over the preregistered analysis cells and is flat in m (2.11/1.99/1.97 bits at $\eta = \frac{1}{2}$). The control writes the same payload into the same write-once output and rewrites nothing, so what separates it from the executed encoder is not irreversibility but whether the prefix can be applied before the suffix is read. The 6:1 ratio of figure 2 is mostly codec slack: under the ranking encoder, which writes only what an executed prefix must determine, the ratio is at most 2.64, and only the slack grows with address width.

Two preregistered attempts to carry the toll onto third-party compilers failed, and we report them as failures. The first, which asked whether toolchains spend committed bytes on addresses, placed every rotation on low-index qubits and so never varied the address width. The second compared circuits that differ only in whether the dispersed subset is an arithmetic progression or a random k-subset, emitted through Qiskit and TKET and compared by compressed size; the excess reached 86% of $\log_2 \binom{m}{k}$ on TKET but peaked at the wrong k, and was within seed noise on Qiskit. A general-purpose compressor may be too coarse an entropy estimator, and neither tool emits a round-major committed stream, so the design premise was not met. Whether the toll is measurable at a third-party interface remains open; details are in the Supplemental Material [30].

6 Reference compiler and external toolchains

We now leave the stream model and report how our reference compiler and external toolchains handle repeated and dispersed layers. The reference compiler is a representation-aware pass in a frozen snapshot of the Unitary Compiler Collection. It recognizes a fixed grammar of high-level forms—explicit Pauli rotations and phase gadgets, local phase-ladder and Fourier spans, mirrored composite blocks, and repeated semantic nodes—keys the terms of each accepted commuting span by generator, adds their coefficients, and emits at most one record per generator, so the output has $O(m)$ phase records independently of the number of repetitions. It does not implement a general symplectic extractor and claims no maximal commuting region. Every selected rewrite must pass an independent equivalence certificate: a dense unitary check for $n \leq 6$, or a sound but incomplete exact symbolic checker that propagates a Clifford tableau through the circuit, extracts the rotation axes, and compares canonical records [30]; failure returns the unchanged candidate. On the supported exact domain, 56 adversarial mutations produce 0 false accepts, and 12 dense–symbolic cross-checks show no disagreement. Only certified `completed_valid` rows enter quality means; unsupported, invalid, and inconclusive outcomes are reported as status.

The external toolchain results are configured-pipeline diagnostics; they report tool behavior and neither prove nor test a lower bound. A matched campaign of 1,080 cells evaluates five compilers on nine representations of the same target—semantic aggregate, contiguous, round-robin, and randomly ordered flat streams, a masked modular-share stream, a locally folded stream, and QASM, TKET, and ZX bridge records—with a common target basis, error budget, timeout, memory cap, and certificate policy; 1,065 cells are `completed_valid` and 15 are retained as `predicate_error`. The matched target uses a fixed modulus and angles that do not wrap, not the ϵ-calibrated grid, and an active term is one with nonzero aggregate. A separate 1,728-cell campaign freezes recommended-native and unified-basis configurations of Qiskit, TKET PauliSimp, PyZX `full_reduce`, and an exact phase-table baseline of the authors, distinct from the reference compiler (1,695 completed, 33 predicate errors). TKET PauliSimp, in its recommended native configuration, and PyZX recover one rotation per active term on the flat representations, so we classify them as semantic-capable: starting from the lowered stream, they rebuild a global intermediate representation in which the terms can be merged, and the bounds charge that representation as state.

On a fixed-width benchmark that repeats one four-qubit diagonal layer—four single-qubit Z rotations and six controlled phases on all pairs—between Hadamard layers (figure 4), our semantic path returns a constant 42 gates and rebased TKET PauliSimp 52–73 gates over five sizes from 4k to 100k requested input gates, while Qiskit's optimization level 3 and staq's rotation folding grow linearly with the input. Disabling the Fourier-layer recognition makes our own compiler grow linearly as well, so the constant output comes from semantic lifting, not from pass order. TKET's GuidedPauliSimp acts only inside `CircBox` regions and returns these flat inputs unchanged in size. The full matrices, the packing-family and width scans, and a real-instance anti-regression panel are in

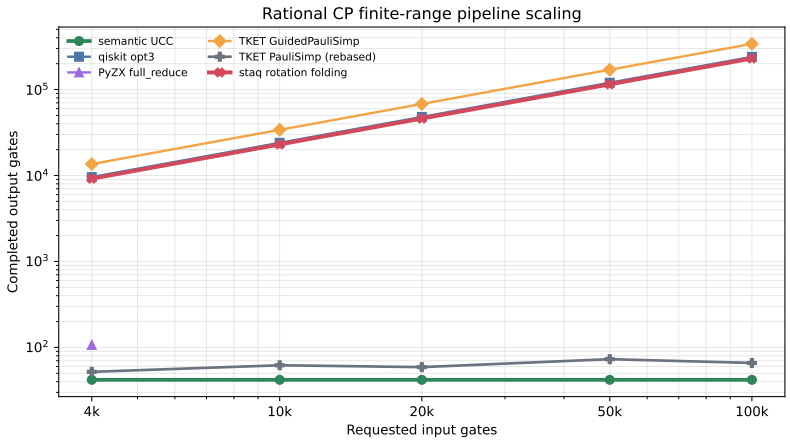


Figure 4: Configured-pipeline scaling on the fixed-width repeated-layer benchmark (HD^rH, with D a four-qubit layer of four single-qubit Z rotations and six controlled phases), against the requested number of input gates. Our semantic path ("semantic UCC") stays at 42 gates and rebased TKET PauliSimp returns 52–73 gates across the five sizes; staq rotation folding and Qiskit's optimization level 3 grow with the input; TKET GuidedPauliSimp returns its input unchanged in size; PyZX `full_reduce` completes at the smallest size and reaches the time budget at larger ones. Status-only cells are omitted from the curves.

the Supplemental Material [30].

A preregistered scan (`PaperDraft/research/E7_preregistration.md`) ran TKET PauliSimp on four-round dispersed streams with modulus 64 on chains of $n \in \{8, \ldots, 128\}$ qubits ($n-1$ terms). It reduced the non-Clifford rotations to one per active term in every cell (508 dispersed rotations to 127 at $n = 128$), although its total output was not compact, and its process-memory increment grew with log–log slope 1.70 over the range selected by the preregistered noise-floor rule, outside the predicted interval $[0.7, 1.5]$; the prediction was therefore rejected. A process-memory increment is not B_{cut}, the scan has no arm on the aggregated layer, and its input text grows with m, so it describes PauliSimp's behavior rather than testing the frontier.

7 Fault-tolerant resource consequences

The bounds count bits; they do not convert into gates or magic states. We therefore propagate the measured outputs of three pipelines through Clifford+T synthesis and surface-code scheduling under one disclosed, versioned model, on the four-qubit repeated-layer benchmark at 4k and 20k requested input gates (399 and 1,999 repetitions of the layer). We call a pipeline semantic-first if the aggregate structure is available before lowering and materialize-first if recovery starts from the lowered stream. The materialize-first pipeline is Qiskit's optimization level 3 (trivial layout, no routing) applied after lowering to $\{cx, rx, ry, rz, h\}$; TKET PauliSimp receives the same circuit lowered to a cx–R_z–h bridge; and the semantic-first reference receives the unlowered circuit. Every pipeline gets the same additive budget: $0.10\epsilon_{\mathrm{total}}$ for the certified algorithmic stage, $0.45\epsilon_{\mathrm{total}}$ for synthesis, and $0.45\epsilon_{\mathrm{total}}$ for logical failure. Every distinct angle is synthesized by a frozen deterministic build of staq `grid_synth` [32, 8], whose reported error never exceeds its allocation across 670 checked angle–precision pairs; exact multiples of $\pi/4$ use staq's checked exact branch. The full matrix crosses two sizes, three pipelines, three total errors, two allocation policies, two surface-code profiles [33, 34, 35, 10] with $p_{\mathrm{phys}} = 10^{-3}$ and 10^{-4}, and 1/4/16/32 factories. All 288 rows are `completed_valid`, carry an exact symbolic or a dense unitary certificate, and meet the requested total error. The accounting follows published end-to-end estimates [36, 37]; the profiles are sensitivity scenarios, not vendor estimates.

Over all 96 matched settings, materialize-first uses 385–1,753 times the logical T states and 538–5,573 times the spacetime volume of semantic-first: 385–501 times the T states at 4k and 1,367–1,753 times at 20k (table 1 and figure 5). The ratio grows with the number of repetitions because the materialize-first output keeps about 12 rotations per repetition (4,792 and 23,992 in total), whereas semantic-first keeps 22 at both sizes. Those 22, four single-qubit terms and three for each controlled phase, merge the repetitions but keep the decomposition of each controlled phase; a full phase-polynomial merge of this layer keeps 10. TKET PauliSimp also merges across that decomposition. Its output has more rotations (29 at 20k), but they include angles at exact multiples of $\pi/4$, which staq synthesizes exactly, and it uses 0.443–0.536 times the semantic-first T states; the comparison does not rank our implementation above tools that rebuild a global representation. Because the inputs differ between pipelines, the ratios measure what is at stake when the repetitions reach synthesis unmerged, not the quality of Qiskit's optimizer. Absolute counts de-

pipeline	N_{rot}	T	physical qubits	factories	cycles	qubit-s
External semantic-capable (TKET)	29	798	258,152	16	13,317	3.44e+03
Materialize-first	23,992	2,679,110	468,968	16	57,966,528	2.72e+07
Semantic-first reference	22	1,800	305,000	16	31,675	9.66e+03

Table 1: Predeclared 20k slice of the fixed-total-error campaign: $\epsilon_{\text{total}} = 10^{-6}$, equal synthesis allocation, the conservative $p_{\text{phys}} = 10^{-3}$ profile, and 16 factories. Every output is certified and every synthesized angle returns staq's `Check flag = 1`.

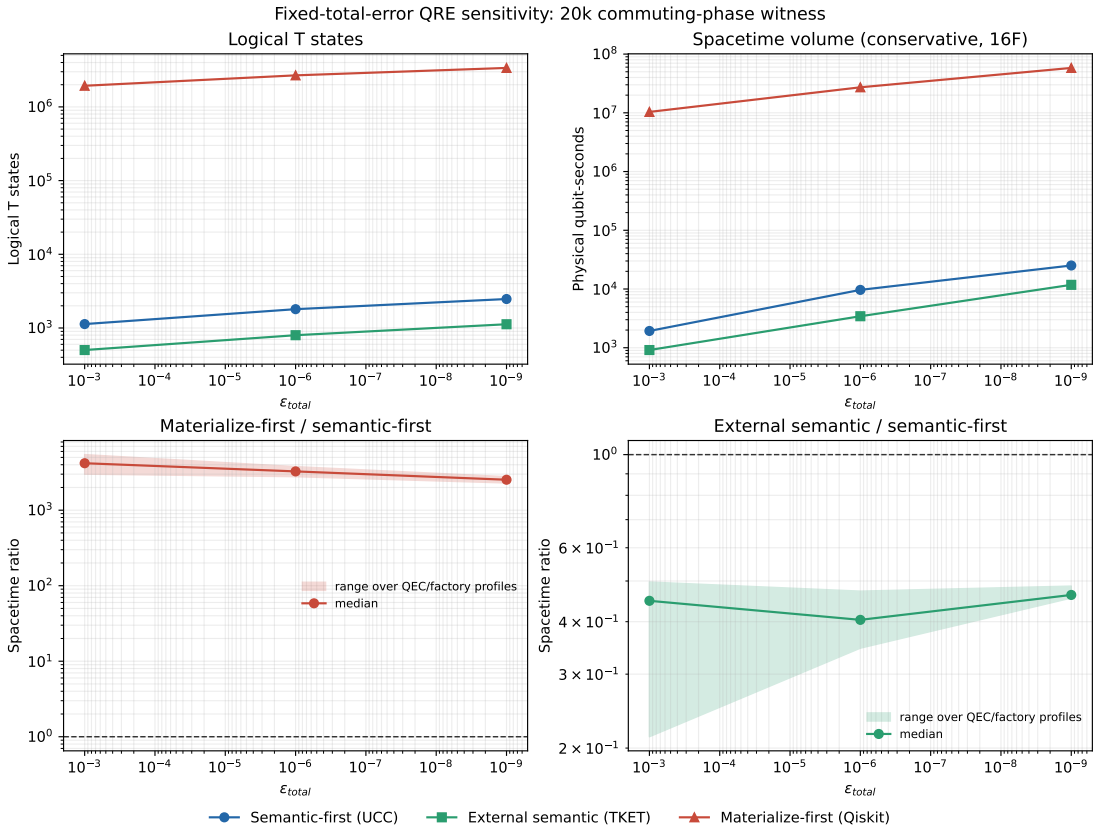


Figure 5: Sensitivity to total error, QEC profile, and factory count at the 20k size. Top: logical T demand and the conservative-profile spacetime volume. Bottom: ratios to semantic-first, with bands spanning both profiles and all factory counts; the dashed line is equality.

pend on routing, correlated errors, factory layout, and feed-forward assumptions outside this campaign.

8 Application workloads

To check that merging carries over to structured circuits and does not regress on circuits without commuting structure, a factorial campaign crosses seven structured families—QAOA/Ising cost evolution [1], diagonal Hamiltonian simulation, commuting-block Trotterization [5], QFT arithmetic, QPE controlled powers, controlled-power ladders, and phase-polynomial blocks [13]—and two negative controls, random Clifford+R_Z and routing-dominated circuits [38, 39], over three scales, three seeds, two densities, and two topologies, following the conventions of [40, 41]. The authors' compiler is evaluated on the full 2^6 matrix of its components (semantic lift, aggregation, selector,

Family	TKET−ours gates [95% CI]	strict
controlled_power_ladder	9.0 [5.7, 12.7]	yes
diagonal_hamiltonian	38.0 [25.9, 51.8]	yes
phase_polynomial_blocks	-24.3 [-33.8, -16.3]	no
qaoa_ising	44.7 [33.7, 56.6]	yes
qft_arithmetic	8.1 [1.9, 13.8]	no
qpe_controlled_powers	-12.0 [-22.0, -3.0]	yes
trotter_commuting_blocks	33.0 [17.6, 48.2]	yes

Table 2: Structured families against TKET PauliSimp. The gate column averages all 36 cells of a family (row-level paired bootstrap); the predeclared strict criterion is evaluated on the nine primary cells (all-to-all topology, density 1) and requires valid outputs, no worse gate and CX counts, and lower compiler runtime at every primary scale and seed. Positive values mean our output has fewer gates; because the strict criterion uses only the primary cells, a family can pass with a negative mean, as `qpe_controlled_powers` does.

cache, preset recognizer, and projected-block selection) and compared with TKET PauliSimp, for 21,060 cells, all certified at six qubits.

Five structured families meet the strict criterion; QFT arithmetic and phase-polynomial blocks do not (table 2). The seeds replicate less than the design suggests: several generators produce the same instance for every seed on the primary cells, so the "every seed" clause is vacuous there. In logical T states, the fault-tolerant metric, the two compilers tie on five families, TKET is better on QAOA/Ising and both controls, and ours only on the Trotter family. Both On both negative controls, switching all six components on changes no metric relative to switching them all off (gates, depth, CX, logical T, spacetime), so the components cause 0 systematic regressions there. The preset recognizer has the largest isolated gate effect, a mean reduction of 364.9 gates (row-level interval [349.8,380.5], which understates the uncertainty because rows share instances); selector, cache, and projected-block selection have none. These workloads repeat identical terms within each block, so they exercise aggregation, not the dispersal bound.

9 Related work

Rotation merging. Commuting-rotation aggregation is a standard optimization target [11, 12, 16, 42, 43, 17, 44]. Phase-polynomial methods make the generator–coefficient structure explicit and resynthesize the parity network [13, 45, 46], and ZX methods keep a global normal form in which phases fuse across syntactic boundaries [47, 48]. These are upper-bound constructions; our bounds go the other way and charge the global representation such tools build as compiler state.

Synthesis and resource estimation. Generic Z-rotations cost $\Theta(\log(1/\epsilon))$ Clifford+T gates [49, 50, 8, 51, 52]. End-to-end estimates propagate non-Clifford counts through distillation and layout [9, 36, 37, 53]; our QRE campaign is an instance of this methodology under a disclosed model, not a new estimator.

Intermediate representations and routing. Quantum software stacks preserve information across lowering in intermediate representations [54, 55, 42, 56, 57, 58]; our model charges what such representations retain as B_{IR}. Routing and platform compilers reshape the stream a downstream stage sees [38, 39, 59, 60, 61]; their SWAP layers normalize away only under the hypothesis of lemma 1.

Noise tailoring. Randomized compiling and Pauli twirling [6, 7, 31] act on the dispersed streams without creating them, and error mitigation [62, 63] shares the relevant feature: the executed circuit is a randomized ensemble whose members do not display the source coefficients.

Lower-bound methodology. The proofs import fooling-set and communication arguments [25, 64] and the space–passes tradition of streaming computation [65, 23, 24, 26]. The commitment toll is related to advice complexity [27, 28] and to relaxed irrevocability [29].

10 Discussion

The results have different scopes. Theorem 2 covers r-round shares, randomized failure, and $p = p(m)$ forward scans re-delivered by the source, and is attained to leading order for p small compared with $m/\log m$; theorem 3 gives the sharper integer form for deterministic one-pass compilers. Reverse scans, uncharged random access to the input, and a different output code are outside both. The commitment toll is the most narrowly scoped result: it needs the prefix to be executed at the cut, it is an ensemble statement about one mask-uniform compiler, and only its record-level form is measured, on our own software model. The physical reading needs contiguous rounds, freely varying coefficients, and a compiler downstream of the schedule and the twirl; we have not exhibited a production pipeline that emits such streams to a bounded downstream stage, and that is the main open question on the applied side. The resource campaign uses actual synthesis and executable surface-code calculations, but under one phenomenological model family and with asymmetric inputs, and its benchmark repeats rather than disperses a layer; the workloads are certified at six qubits, and their size axis repeats structure rather than widening it.

11 Conclusion

A compiler that receives a commuting phase layer whose rounds are contiguous, up to generator-permuting Clifford records, and whose per-round coefficients vary freely, possibly re-signed by randomized compiling, and that reads it in forward passes and is correct with probability at least $1 - \delta$ on every such stream, must hold or have committed $(1 - \delta)m \log_2 K_\epsilon - h_2(\delta)$ bits, about $\log_2(1/\epsilon)$ per independent term. At $\delta = 0$ a one-pass compiler that keeps the running sums, or a p-pass compiler that merges one block of terms per pass, meets that price to leading order. The two channels part ways when the committed first-round rotations are applied at the cut: the prefix must then pay for the dispersal pattern, the full entropy if it addresses its targets and a smaller but linear amount if it only implements them, while buffered output pays nothing. Under one disclosed

synthesis and surface-code model, Qiskit's highest preset optimization level, given the pre-lowered circuit, needs 385–1,753 times the logical T states of our compiler's output from the unlowered circuit, a ratio that grows with the number of repetitions, while TKET's PauliSimp, which rebuilds the phase structure, needs about half as many as ours; the inputs differ, so these ratios measure what is at stake when repetitions reach synthesis unmerged. The cost of recovering merged rotations from a dispersed stream is thus a quantity that a software stack can budget for: where the interface allows, randomize and lower late, and keep phase semantics until the coefficients have been merged.

Reproducibility

Every data-derived number, table, and figure is generated from registered frozen sources (`data/manifest.yaml`) by `scripts/build_all_tables.py` and `scripts/build_all_figures.py`; the source row and operation of each printed number are declared in `data/provenance_map.yaml` and tested by `tests/test_paper_number_provenance.py`. The one-command rebuild is `reproducibility/run_submission.sh`; it reproduces the analysis from the frozen data but does not rerun the measurement campaigns. The campaign protocols, status accounting, and data lineage are in the Supplemental Material [30].

Acknowledgements

J.Y.'s deepest thanks go to two teachers who are also coauthors of this paper, Yangyang Li and Xiu-Hao Deng. Their help was immense; without them this paper would not exist.

J.Y. thanks Ke Lin of Xidian University for opening the door to research and for the recommendation that led him to join Prof. Li's group.

J.Y. thanks Brad Chase: the first ideas behind this paper arose while fixing an issue in his `ucc` repository.

J.Y. also thanks the teachers who were willing to offer guidance along the way, and the senior members of Prof. Li's group, who have been generous with their help throughout this work.

Thanks are due, finally, to everyone who supported this paper, too many to name individually here.

Y.L. acknowledges support from the National Natural Science Foundation of China under Grant No. 62476209, the Key Research and Development Program of Shaanxi under Grant No. 2024CY2-GJHX-18, and the Fundamental Research Funds for the Central Universities under Grant No. QTZX26097. X.-H.D. acknowledges support from the Science, Technology and Innovation Commission of Shenzhen Municipality (Grant Nos. JCYJ20170412152620376 and KYTDPT20181011104202253), the Innovation Program for Quantum Science and Technology (Grant No. 2021ZD0301703), the Guangdong Major Project of Basic Research (Grant No. 2025B0303000007), and the Shenzhen Science and Technology Program (Grant No. KQTD20200820113010023).

This paper pays particular attention to provenance: every printed number is traceable to a unique source row, every experiment to a frozen registry, and every preregistered prediction to a criterion written down before the run. That being so—*what one begins with, one must end with* (*Zuo Zhuan*, Xuan 12)—it is fitting to record the provenance of the framing as well, even though that provenance does not lie in the physics literature.

Two passages from the classical Chinese canon shaped parts of this work. Their relevance is direct, and the acknowledgements provide a fitting context in which to quote them.

The first is from the *Xici* commentary to the *Yijing*: "What is above form is called the Way; what is below form is called the vessel; to transform and tailor it is called change; to carry it forward and put it into practice is called continuity." What is useful here is not an analogy but a set of distinctions. The passage names four things separately, and ranks none of them above another: a description that carries structure, a description that carries only instructions, the act of lowering the one into the other, and the act of execution. A compiler is precisely the engine that performs the third and hands off the fourth; and the vocabulary of compilation itself—semantic level, lowering, commitment—reproduces the same fourfold division without, so far as we know, having borrowed it. The lesson we draw from the passage is that the level at which a computation is described is not a notational convenience to be optimized away, but a variable that can be charged for. This paper charges it, in bits.

The second is Chapter 64 of the *Laozi*: "What is at rest is easy to hold; what has not yet shown a sign is easy to plan for; what is brittle is easy to shatter; what is minute is easy to scatter. Act on it before it comes to be; order it before it falls into disorder." Every clause here prices an action by the state of the object acted upon rather than by the action itself—which is exactly this paper's claim about representation: the same aggregation is cheap or expensive depending only on the form in which the layer arrives. The verb of the first clause, *to hold*, is the operation charged here as compiler state. The chapter further separates planning early, which it recommends, from grasping early, warning that "one who grasps it loses it"; that separation is, here, the difference between a compiler that defers and one that commits, and the price of the difference is the commitment toll. And the rule on which the chapter closes is the design rule of our abstract, read from the other side.

Neither passage contains a theorem, and neither an-

ticipates one; no result reported here depends on either. A reader who finds these readings strained may discard them entirely and judge the paper by whatever in it proves useful. They are recorded because the question did come from there and because, for a framing as for a datum, a reader may rightly ask where it came from. Translations are J.Y.'s own.

Generative AI tools, including large language model assistants such as Claude (Anthropic), were used to assist with manuscript drafting and revision, code and repository maintenance, literature discovery, and consistency checks. The authors directed their use through task-specific prompts and independently reviewed the generated text, citations, code, and numerical claims against the cited literature, source files, executable tests, and the symbolic or dense certificates reported in this work. AI output was not treated as a scientific source or as independent proof verification. The authors take full responsibility for the manuscript, proofs, code, data, figures, citations, and conclusions.

Author contributions

J.Y. developed the theory and proofs, implemented the compiler instrumentation and experimental pipeline, and performed the majority of the measurements and analyses. Y.L. supervised the research, reviewed and revised the manuscript, and provided project administration and funding support. X.-H.D. independently validated the theoretical derivations, numerical results, and computational artifacts. All authors wrote the manuscript together, discussed the results, and approved the final version.

References

[1] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. "A quantum approximate optimization algorithm" (2014). arXiv:1411.4028.

[2] Rebekah Herrman, Phillip C. Lotshaw, James Ostrowski, Travis S. Humble, and George Siopsis. "Multi-angle quantum approximate optimization algorithm". Scientific Reports **12**, 6781 (2022). arXiv:2109.11455.

[3] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O'Brien. "A variational eigenvalue solver on a photonic quantum processor". Nature Communications **5**, 4213 (2014).

[4] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. "Variational quantum algorithms". Nature Reviews Physics **3**, 625–644 (2021).

[5] Andrew M. Childs, Yuan Su, Minh C. Tran, Nathan Wiebe, and Shuchen Zhu. "Theory of Trotter error with commutator scaling". Physical Review X **11**, 011020 (2021).

[6] Joel J. Wallman and Joseph Emerson. "Noise tailoring for scalable quantum computation via randomized compiling". Physical Review A **94**, 052325 (2016). arXiv:1512.01098.

[7] Akel Hashim, Ravi K. Naik, Alexis Morvan, Jean-Loup Ville, Bradley Mitchell, John Mark Kreikebaum, Marc Davis, Ethan Smith, Costin Iancu, Kevin P. O'Brien, Ian Hincks, Joel J. Wallman, Joseph Emerson, and Irfan Siddiqi. "Randomized compiling for scalable quantum computing on a noisy superconducting quantum processor". Physical Review X **11**, 041039 (2021). arXiv:2010.00215.

[8] Neil J. Ross and Peter Selinger. "Optimal ancilla-free Clifford+T approximation of Z-rotations". Quantum Information and Computation **16**, 901–953 (2016). arXiv:1403.2975.

[9] Sergey Bravyi and Alexei Kitaev. "Universal quantum computation with ideal Clifford gates and noisy ancillas". Physical Review A **71**, 022316 (2005).

[10] Daniel Litinski. "Magic state distillation: Not as costly as you think". Quantum **3**, 205 (2019). arXiv:1905.06903.

[11] Matthew Amy, Dmitri Maslov, and Michele Mosca. "Polynomial-time T-depth optimization of Clifford+T circuits via matroid partitioning". IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **33**, 1476–1489 (2014).

[12] Yunseong Nam, Neil J. Ross, Yuan Su, Andrew M. Childs, and Dmitri Maslov. "Automated optimization of large quantum circuits with continuous parameters". npj Quantum Information **4**, 23 (2018).

[13] Matthew Amy, Parsiad Azimzadeh, and Michele Mosca. "On the controlled-NOT complexity of controlled-NOT–phase circuits". Quantum Science and Technology **4**, 015002 (2018).

[14] Matthew Amy and Joseph Lunderville. "Linear and non-linear relational analyses for quantum program optimization". Proceedings of the ACM on Programming Languages **9**, 1072–1103 (2025).

[15] Aws Albarghouthi. "Linear-time T-gate optimization via random abstraction" (2026). arXiv:2605.13929.

[16] Alexander Cowtan, Silas Dilkes, Ross Duncan, Will Simmons, and Seyon Sivarajah. "Phase gadget synthesis for shallow circuits". In Proceedings 16th International Conference on Quantum Physics and Logic (QPL 2019). Volume 318 of Electronic Proceedings in Theoretical Computer Science, pages 213–228. (2020). arXiv:1906.01734.

[17] Aleks Kissinger and John van de Wetering. “Reducing the number of non-Clifford gates in quantum circuits”. Physical Review A **102**, 022406 (2020). arXiv:1903.10477.

[18] John van de Wetering, Richie Yeung, Tuomas Laakkonen, and Aleks Kissinger. “Optimal compilation of parametrised quantum circuits”. Quantum **9**, 1828 (2025).

[19] Craig Gidney. “Halving the cost of quantum addition”. Quantum **2**, 74 (2018).

[20] Ian D. Kivlichan, Craig Gidney, Dominic W. Berry, Nathan Wiebe, Jarrod McClean, Wei Sun, Zhang Jiang, Nicholas Rubin, Austin Fowler, Alán Aspuru-Guzik, Hartmut Neven, and Ryan Babbush. “Improved fault-tolerant quantum simulation of condensed-phase correlated electrons via trotterization”. Quantum **4**, 296 (2020).

[21] Andrew Cross, Ali Javadi-Abhari, Thomas Alexander, Niel de Beaudrap, Lev S. Bishop, Steven Heidel, Colm A. Ryan, Prasahnt Sivarajah, John Smolin, Jay M. Gambetta, and Blake R. Johnson. “OpenQASM 3: A broader and deeper quantum assembly language”. ACM Transactions on Quantum Computing **3**, 12 (2022).

[22] Leon Riesebos, Xiang Fu, Savvas Varsamopoulos, Carmen G. Almudever, and Koen Bertels. “Pauli frames for quantum computer architectures”. In Proceedings of the 54th Annual Design Automation Conference (DAC 2017). Pages 76:1–76:6. (2017).

[23] Noga Alon, Yossi Matias, and Mario Szegedy. “The space complexity of approximating the frequency moments”. Journal of Computer and System Sciences **58**, 137–147 (1999).

[24] Sudipto Guha and Andrew McGregor. “Tight lower bounds for multi-pass stream computation via pass elimination”. In Automata, Languages and Programming (ICALP 2008). Pages 760–772. Lecture Notes in Computer Science. Springer (2008).

[25] Eyal Kushilevitz and Noam Nisan. “Communication complexity”. Cambridge University Press. (1997).

[26] Nathanaël François, Rahul Jain, and Frédéric Magniez. “Unidirectional input/output streaming complexity of reversal and sorting”. In Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2014). Volume 28 of Leibniz International Proceedings in Informatics (LIPIcs), pages 654–668. (2014). arXiv:1309.0647.

[27] Hans-Joachim Böckenhauer, Juraj Hromkovič, Dennis Komm, Sacha Krug, Jasmin Smula, and Andreas Sprock. “The string guessing problem as a method to prove lower bounds on the advice complexity”. Theoretical Computer Science **554**, 95–108 (2014).

[28] Joan Boyar, Lene M. Favrholdt, Christian Kudahl, Kim S. Larsen, and Jesper W. Mikkelsen. “Online algorithms with advice: A survey”. ACM Computing Surveys **50**, 1–34 (2017).

[29] Joan Boyar, Lene M. Favrholdt, Michal Kotrbčík, and Kim S. Larsen. “Relaxing the irrevocability requirement for online graph algorithms”. Algorithmica **84**, 1916–1951 (2022).

[30] See Supplemental Material for the record codec and field-level cut accounting, the certificate domain and its soundness proof, the exact-semantics results for fixed-width circuits, the complete experimental protocols, matrices, and negative results, and the data lineage.

[31] Emanuel Knill. “Quantum computing with realistically noisy devices”. Nature **434**, 39–44 (2005).

[32] Matthew Amy and Vlad Gheorghiu. “staq—a full-stack quantum processing toolkit”. Quantum Science and Technology **5**, 034016 (2020). arXiv:1912.06070.

[33] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. “Surface codes: Towards practical large-scale quantum computation”. Physical Review A **86**, 032324 (2012).

[34] Dominic Horsman, Austin G. Fowler, Simon Devitt, and Rodney Van Meter. “Surface code quantum computing by lattice surgery”. New Journal of Physics **14**, 123011 (2012).

[35] Daniel Litinski. “A game of surface codes: Large-scale quantum computing with lattice surgery”. Quantum **3**, 128 (2019). arXiv:1808.02892.

[36] Craig Gidney and Martin Ekerå. “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits”. Quantum **5**, 433 (2021). arXiv:1905.09749.

[37] Michael E. Beverland, Prakash Murali, Matthias Troyer, Krysta M. Svore, Torsten Hoefler, Vadym Kliuchnikov, Guang Hao Low, Mathias Soeken, Aarthi Sundaram, and Alexander Vaschillo. “Assessing requirements to scale to practical quantum advantage” (2022). arXiv:2211.07629.

[38] Alexander Cowtan, Silas Dilkes, Ross Duncan, Alexandre Krajenbrink, Will Simmons, and Seyon Sivarajah. “On the qubit routing problem”. In 14th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2019). Volume 135 of Leibniz International Proceedings in Informatics, pages 5:1–5:32. (2019).

[39] Gushu Li, Yufei Ding, and Yuan Xie. “Tackling the qubit mapping problem for NISQ-era quantum devices”. In Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2019). Pages 1001–1014. (2019).

[40] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. “QASMBench: A low-level quantum benchmark suite for NISQ evaluation

and simulation". ACM Transactions on Quantum Computing **4**, 10 (2023).

[41] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. "MQT Bench: Benchmarking software and design automation tools for quantum computing". Quantum **7**, 1062 (2023). arXiv:2204.13719.

[42] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. "t|ket⟩: A retargetable compiler for NISQ devices". Quantum Science and Technology **6**, 014003 (2021).

[43] Ross Duncan, Aleks Kissinger, Simon Perdrix, and John van de Wetering. "Graph-theoretic simplification of quantum circuits with the ZX-calculus". Quantum **4**, 279 (2020). arXiv:1902.03178.

[44] Aleks Kissinger and John van de Wetering. "PyZX: Large scale automated diagrammatic reasoning". In Proceedings 16th International Conference on Quantum Physics and Logic (QPL 2019). Volume 318 of Electronic Proceedings in Theoretical Computer Science, pages 229–241. (2020).

[45] Niel de Beaudrap, Xiaoning Bian, and Quanlong Wang. "Techniques to reduce $\pi/4$-parity-phase circuits, motivated by the ZX calculus". In Proceedings 16th International Conference on Quantum Physics and Logic (QPL 2019). Volume 318 of Electronic Proceedings in Theoretical Computer Science, pages 131–149. (2020).

[46] Arianne Meijer-van de Griend and Ross Duncan. "Architecture-aware synthesis of phase polynomials for NISQ devices". In Proceedings 19th International Conference on Quantum Physics and Logic (QPL 2022). Volume 394 of Electronic Proceedings in Theoretical Computer Science, pages 116–140. (2023).

[47] John van de Wetering. "ZX-calculus for the working quantum computer scientist" (2020). arXiv:2012.13966.

[48] Aleks Kissinger and John van de Wetering. "Simulating quantum circuits with ZX-calculus reduced stabiliser decompositions". Quantum Science and Technology **7**, 044001 (2022).

[49] Christopher M. Dawson and Michael A. Nielsen. "The Solovay–Kitaev algorithm". Quantum Information and Computation **6**, 81–95 (2006).

[50] Vadym Kliuchnikov, Dmitri Maslov, and Michele Mosca. "Fast and efficient exact synthesis of single-qubit unitaries generated by Clifford and T gates". Quantum Information and Computation **13**, 607–630 (2013).

[51] Peter Selinger. "Efficient Clifford+T approximation of single-qubit operators". Quantum Information and Computation **15**, 159–180 (2015).

[52] Vlad Gheorghiu, Michele Mosca, and Priyanka Mukhopadhyay. "T-count and T-depth of any multi-qubit unitary". npj Quantum Information **8**, 141 (2022).

[53] Ryan Babbush, Jarrod R. McClean, Michael Newman, Craig Gidney, Sergio Boixo, and Hartmut Neven. "Focus beyond quadratic speedups for error-corrected quantum advantage". PRX Quantum **2**, 010103 (2021).

[54] Damian S. Steiger, Thomas Häner, and Matthias Troyer. "ProjectQ: An open source software framework for quantum computing". Quantum **2**, 49 (2018). arXiv:1612.08091.

[55] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. "Quantum computing with Qiskit" (2024). arXiv:2405.08810.

[56] George Watkins, Hoang Minh Nguyen, Keelan Watkins, Steven Pearce, Hoi-Kwan Lau, and Alexandru Paler. "A high performance compiler for very large scale surface code computations". Quantum **8**, 1354 (2024). arXiv:2302.02459.

[57] Anurudh Peduri, Siddharth Bhat, and Tobias Grosser. "QSSA: An SSA-based IR for quantum computing". In Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction (CC 2022). Pages 2–14. (2022).

[58] Alexander McCaskey and Thien Nguyen. "A MLIR dialect for quantum assembly languages". In 2021 IEEE International Conference on Quantum Computing and Engineering (QCE). Pages 255–264. (2021).

[59] Simon Martiel and Timothée Goubault de Brugière. "Architecture aware compilation of quantum circuits via lazy synthesis". Quantum **6**, 729 (2022). arXiv:2012.09663.

[60] Fabian Kreppel, Christian Melzer, Diego Olvera Millán, Janis Wagner, Janine Hilder, Ulrich Poschinger, Ferdinand Schmidt-Kaler, and André Brinkmann. "Quantum circuit compiler for a shuttling-based trapped-ion quantum computer". Quantum **7**, 1176 (2023). arXiv:2207.01964.

[61] Daniel Bochen Tan, Dolev Bluvstein, Mikhail D. Lukin, and Jason Cong. "Compiling quantum circuits for dynamically field-programmable neutral atoms array processors". Quantum **8**, 1281 (2024). arXiv:2306.03487.

[62] Zhenyu Cai, Ryan Babbush, Simon C. Benjamin, Suguru Endo, William J. Huggins, Ying Li, Jarrod R. McClean, and Thomas E. O'Brien. "Quantum error mitigation". Reviews of Modern Physics **95**, 045005 (2023).

[63] Ewout van den Berg, Zlatko K. Minev, Abhinav Kandala, and Kristan Temme. "Probabilistic error cancellation with sparse Pauli–Lindblad models on noisy quantum processors". Nature Physics **19**, 1116–1121 (2023).

[64] Michael Sipser. "Introduction to the theory of computation". Cengage Learning. (2012). 3 edition.

[65] J. Ian Munro and Michael S. Paterson. "Selection and sorting with limited storage". Theoretical Computer Science **12**, 315–323 (1980).

[66] John Watrous. "The theory of quantum information". Cambridge University Press. Cambridge (2018).

A Proofs

A.1 Projective and diamond distance

Lemma 2 (Unitary-channel diamond comparison)**.** *For the unitary channels $\mathcal{U}(X) = UXU^\dagger$ and $\mathcal{V}(X) = VXV^\dagger$, define the* unnormalized *diamond distance $d_\diamond(\mathcal{U},\mathcal{V}) := \|\mathcal{U}-\mathcal{V}\|_\diamond \in [0,2]$. Then*

$$d_{\mathrm{proj}}(U,V) \le d_\diamond(\mathcal{U},\mathcal{V}) \le 2d_{\mathrm{proj}}(U,V).$$

If the normalized convention $\widetilde{d}_\diamond := \frac{1}{2}\|\mathcal{U}-\mathcal{V}\|_\diamond$ is used, the corresponding explicit relation is $\frac{1}{2}d_{\mathrm{proj}} \le \widetilde{d}_\diamond \le d_{\mathrm{proj}}$.

Proof. Let $\Theta \in [0,2\pi]$ be the length of a shortest closed arc containing the spectrum of $U^\dagger V$. Centering that arc by a global phase puts every eigenvalue within angle $\Theta/2$ of 1; conversely, for every phase the rotated spectrum still spans an arc of length Θ, so some eigenvalue lies at angle at least $\Theta/2$ from 1. Hence

$$d_{\mathrm{proj}}(U,V) = 2\sin(\Theta/4).$$

The numerical-range formula for two unitary channels [66] gives

$$d_\diamond(\mathcal{U},\mathcal{V}) = \begin{cases} 2\sin(\Theta/2), & 0 \le \Theta \le \pi, \\ 2, & \pi \le \Theta \le 2\pi. \end{cases}$$

For $\Theta \le \pi$, the ratio is $2\cos(\Theta/4) \in [\sqrt{2},2]$; for $\Theta \ge \pi$, one has $d_{\mathrm{proj}} \in [\sqrt{2},2]$ and $d_\diamond = 2$. The two inequalities follow. □

A.2 The accuracy grid

Proof of corollary 1. At $\epsilon = 1/8$ one has $Q_\epsilon = 6$, so $Q_\epsilon \ge 6$ on the whole range. The bounds $2\epsilon \le \arcsin(2\epsilon) \le \pi\epsilon$ and $\lfloor a \rfloor \ge a-1$ give $1/(4\epsilon) \le 1/(2\epsilon)-1 \le K_\epsilon \le \pi/(4\epsilon)$. Since $Q_\epsilon \le \pi/[2\arcsin(2\epsilon)]$, we have $\pi/(2Q_\epsilon) \ge \arcsin(2\epsilon)$, and theorem 1 with $K = Q_\epsilon$ gives the separation. For the count, fix a non-identity Pauli P. Both characters ± 1 occur, so, up to a global sign, $R_P(\theta)R_P(\theta')^\dagger$ has eigenvalue arguments $\mp g/2$, where $g \in [0,\pi]$ is the circular distance between θ and θ' in $\mathbb{R}/2\pi\mathbb{Z}$, and centering gives $d_{\mathrm{proj}} = 2\sin(g/4)$. With $g_\epsilon := 4\arcsin(2\epsilon)$ one has $2\sin(g_\epsilon/4) = 4\epsilon$, so pairwise 4ϵ-separation is pairwise circular distance at least g_ϵ. Points on a circle of circumference 2π with pairwise gaps at least g_ϵ number at most $\lfloor 2\pi/g_\epsilon \rfloor$, and equal spacing attains this; finally $\lfloor 2\pi/g_\epsilon \rfloor = \lfloor \pi/(2\arcsin(2\epsilon)) \rfloor = Q_\epsilon$. □

A.3 Output-code modes

The codes of definition 3 are fixed prefix-free streaming binary codes. E_{self} serializes a target-basis circuit with gate opcodes, all qubit and generator addresses, record delimiters, and sparse parameters, and assumes no dictionary. Given $\mathcal{F}_m = (m, P_1, \dots, P_m, K, \Delta)$, $E_{\mathrm{rel}}(\cdot \mid \mathcal{F}_m)$ has four modes. (i) Aggregate mode: a self-delimiting header declaring a block length b, followed by the m aggregate digits packed in base K in consecutive blocks of b digits, each written least significant bit first as a $\lceil b\log_2 K \rceil$-bit integer, and a final block of $b' < b$ digits in $\lceil b'\log_2 K \rceil$ bits. (ii) Literal mode: complete target-basis records. (iii) Ordered-share mode: a header declaring a split point, after which the fixed public schedule lets generator addresses be omitted from the residue slots, packed in base Q, while round-boundary and packed-block delimiters are retained. (iv) p-block hybrid mode: a header declaring the pass blocks and passthrough coordinates, followed by packed aggregate blocks and, for each round, the passthrough residues of that round packed in base Q, in the order in which one left-to-right scan per pass produces them, a public function of (m,p,q,r).

A.4 Upper bounds

Proof of proposition 2. Partition the first q coordinates into at most p consecutive blocks of size at most h. Each pass is re-delivered by the input source (definition 2), so the compiler keeps no copy of the stream between passes. On pass t, keep only the residues of block t and add their first $r-1$ rounds modulo Q; when the final-round mates arrive, emit the corresponding aggregate base-Q block. The partial sums are held as one packed integer in the store field at its literal bit length (section 2.2); the live packed block, indices, and framing give (13) at the crossings. Each block is packed separately, which costs at most one bit per block; together with the per-block framing this is the $O(p\log(mQrp))$ term in the output bound. On the first pass, emit every share of each unselected coordinate in the declared round-major order, packing the residues of each round in base Q and flushing them at the end of that round, so that the residues of round $r-1$ are written before the crossing; these rotations already have the correct product. The selected coordinates can be balanced across the p blocks, so the maximum block has size $\lceil q/p \rceil$. Their consecutive boundaries are a public function of (m,p,q), so the header stores those integers rather than p separate delimiters. The prefix contains $r-1$ residues and the whole output contains r residues for each such coordinate, giving (14) and (15).

If $ph \geq m$ there are no passthrough coordinates, A_p is the header, and $p\lceil m/p \rceil \log_2 Q \leq m \log_2 Q + p \log_2 Q$ gives the stated bound on $A_p + pS$ for any r. For $r = 2$ and general q, $(m-q)\log_2 Q + p\lceil q/p \rceil \log_2 Q$ is at most $m \log_2 Q + p \log_2 Q$, which gives the same bound; since every forward snapshot is bounded by the same block, the bound also holds for $A_p + \Sigma_p^{\rightarrow}$. □

Proof of proposition 3. Buffer the incoming aggregates b at a time, held as one packed integer, and write each full buffer as one $\lceil b \log_2 K \rceil$-bit integer and a final partial block of $b' < b$ digits in $\lceil b' \log_2 K \rceil$ bits; the buffer, an index, and framing are the only live state. Each block loses less than one bit to rounding, which gives the $\lceil m/b \rceil$ term. For the single packed integer, let $P = \sum_{i \leq j} x_i K^{i-1}$ be the value of the first j digits and $N = P + K^j R$ the final integer. Since $K^j R \equiv 0 \pmod{2^{ej}}$ and the higher bits of N depend on R, a correct write-once writer has written at most the low ej bits of P, so at most 2^{ej+1} distinct written prefixes occur. Two digit prefixes with the same snapshot and the same written prefix produce the same N for every R, hence $P = P'$. The K^j values of P therefore need at least $K^j/2^{ej+1} = K_{\text{odd}}^j/2$ distinct snapshots, which, being self-delimiting (section 2.2), need at least $j \log_2 K_{\text{odd}} - 1$ bits by Kraft's inequality; take $j = \lfloor m/2 \rfloor$. Proposition 1 gives semantic equality with the flat representation. Prefix the output with the abort-coin branch tag for the randomized construction and take expectations. The self-contained construction writes each nonzero $R_{P_j}(2\pi x_j/Q)$ with its address and rational parameter. □

A.5 Physical streams

Proof of lemma 1. Write the emitted unitary as $D'_r L_r \cdots D'_1 L_1$ with L_t the round-t layer and D'_t the Clifford record following it, so $D_t = D'_t \cdots D'_1$. Moving each Clifford leftwards replaces L_t by $D_{t-1}^\dagger L_t D_{t-1}$ and leaves D_r as a suffix. Since $L_t = \prod_j R_{P_j}(\Delta s_j^{(t)} a_j^{(t)})$ and conjugation is an automorphism, the hypothesis gives $D_{t-1}^\dagger L_t D_{t-1} = \prod_j R_{P_{\pi_t(j)}}(\Delta \varepsilon_j^{(t)} s_j^{(t)} a_j^{(t)})$, which is a round of a scheduled stream after the stated reindexing. The Clifford records lie in the stream, so D_{t-1}, and hence π_t and $\varepsilon^{(t)}$, are computable from the prefix. Finally D_r is common to every member of the family and unitary, so it preserves projective distance and the separation of theorem 1 is unaffected. □

Proof of proposition 4. In (a) and (b) the stated coefficient-freedom hypothesis makes the round coefficients unconstrained across rounds, so every residue sequence—in particular every valid share decomposition of definition 1—arises from some schedule, and proposition 1 supplies realizability and token privacy verbatim. For prefixes: after any prefix that ends before the last round, a final-round residue choice reaches every aggregate in $\mathbb{Z}_Q^m$. In (c), sign normalization $a \mapsto sa$ is a bijection of $\mathbb{Z}_Q$ for either sign, so normalizing every record maps the signed family onto the unsigned family record-for-record, and the same properties transfer; the distributional clause is the one proved in proposition 1, applied after normalization. For the constant schedule, $x_j = ra_j \bmod Q$ is a function of the first round. An exhaustive enumeration at $(m, r, Q) = (2, 2, 5)$, frozen at `data/frozen/fooling_set_m2_r2_Q5.json`, independently confirms all three behaviors: every time-varying and every signed prefix class reaches all Q^m aggregates, and each constant-schedule prefix class reaches exactly one. □

Proof of proposition 5. (i) Sign normalization is a bijection of $\mathbb{Z}_Q$ for either sign, so normalizing every record maps the twirled family onto the family of definition 1 while preserving aggregates, record boundaries, and cut positions; each normalized stream is the image of the twirled streams that differ only in their sign patterns. The cited theorems apply to the image and pull back verbatim.

(ii) Take prefixes whose residue fields emit the residue 1 exactly once on each generator and are otherwise zero, and whose sign fields realize the frame class $\sigma \in \{\pm 1\}^m$; the residue fields are identical across classes. Choose one common suffix adding a residue a_j on generator j, for instance $a_j = 0$. The completed aggregates $x_j = \sigma_j + a_j \bmod Q$ then differ between two classes in every coordinate where the frames differ, because $2 \not\equiv 0 \pmod{Q}$ for $Q \geq 3$, and all completions are valid family instances. By theorem 1 the corresponding targets are separated by $2\sin(\pi/(2Q)) > 2\epsilon$. For a deterministic one-pass compiler the suffix computation is a function of the snapshot and the common suffix; a run that halted before c would give the same output for two classes with different targets, so every run reaches c, and the pair (prefix output, snapshot) must be injective on the 2^m frame classes; prefix-freeness of the self-delimiting snapshot followed by the committed prefix and Kraft's inequality give the deterministic bound. For the multipass statement, draw σ uniformly and repeat the forward-transcript argument of theorem 2 with the common suffix in place of V: the forward transcript determines the final output, the decoder recovers σ with error at most δ, and Fano's inequality gives $I(\sigma; T) \geq (1-\delta)m - h_2(\delta)$. The frame register is classical restart state, so its bits lie in the named fields of the snapshot.

In folded mode the sign is consumed at emission, so the crossing information consists of the signed residues ± 1 themselves, which are distinct for $Q \geq 3$, and the argument applies verbatim to the folded records. □

Proof of proposition 6. Condition on the mask T. Restrict the signs to $s \equiv +1$ and fix the exposed coordinates; T and the exposed values are then public, known to both Alice and Bob in the transcript argument, and are built into the transition function

rather than stored as state. The targets are the dispersed family on the $|T|$ generators in T multiplied by one fixed unitary W of the exposed coordinates. The proof of theorem 2 uses only pairwise separation of the targets, which this translation preserves, so it applies unchanged and gives $\overline{A}_p(T) + pS(T) \geq (1-\delta)|T| \log_2 K - h_2(\delta) - \lceil \log_2(p+1) \rceil$ for every fixed T. Take the expectation over the Bernoulli mask and use $\mathbb{E}|T| = \eta m$. The one-pass clause follows in the same way from theorem 3 applied to the dispersed coordinates. □

A.6 Commitment toll

Proof of corollary 2. For $j \in T$ the signed residue $c_j := s_j^{(1)} a_j^{(1)} \bmod Q$ is uniform on $\mathbb{Z}_Q$, because $a_j^{(1)}$ is uniform and independent of $s_j^{(1)}$; so (T, c) is uniform on $\binom{[m]}{k} \times \mathbb{Z}_Q^k$. If $y \neq y'$, the corresponding round-one actions differ by a nonzero multiple of $\Delta = 2\pi/Q$ on some generator, and theorem 1 with the full residue alphabet $K = Q$ separates them by $2\sin(\pi/(2Q)) > 2\epsilon$; integer and modular residues give the same rotations up to a global phase ± 1. No prefix is within ϵ of both, so an executed prefix determines y, and mask uniformity makes the prefixes one prefix-free code; source coding gives $\mathbb{E}[A_{\text{pre}}] \geq H(y)$. The map $(T, c) \mapsto (y, T)$ is injective, so $H(y) = H(T, c) - H(T \mid y)$. Given y with support of size ν, the consistent masks are the k-sets containing the support, all equally likely, so $H(T \mid y) = \mathbb{E} \log_2 \binom{m-\nu}{k-\nu}$ with $\nu = |\operatorname{supp} y| \sim \text{Bin}(k, 1-1/Q)$; this is (24). Equivalently, writing $V := \operatorname{supp} y$, the values of y on V are uniform on $(\mathbb{Z}_Q \setminus \{0\})^V$ and independent of T given V, so $H(T \mid y) = H(T \mid V)$ and the excess of $H(y)$ over $k \log_2 Q$ is $H(T) - H(T \mid V) = I(T; V)$. The deferred compiler stores the k signed residues in the public round-major order and recovers T from the round-two record types, as in proposition 7(i); the target depends on the signs only through the signed aggregate (20), so the folded residues suffice. Subtracting the deferred state gives the toll. For (25), use $I(T; V) = H(V) - H(V \mid T)$. Given T, each member of T lies in V independently with probability $1 - 1/Q$, so $H(V \mid T) = k\, h_2(1/Q)$. Each $j \in [m]$ lies in V with probability $\eta(1 - 1/Q)$, so subadditivity gives $H(V) \leq m\, h_2(\eta(1 - 1/Q))$. Conversely V is a uniformly random subset of $[m]$ given its size $\nu \sim \text{Bin}(k, 1 - 1/Q)$, so $H(V) \geq \mathbb{E} \log_2 \binom{m}{\nu}$. The bound $\log_2 \binom{m}{s} \geq m\, h_2(s/m) - \log_2(m+1)$, a second-order expansion of h_2 around $\eta(1-1/Q)$, which lies in $(0, 1)$, and a Chernoff tail bound for ν give $H(V) \geq m\, h_2(\eta(1 - 1/Q)) - O(\log m)$. Dividing by $k = \eta m$ gives the limit, approached from below. For attainability, given ν the support is uniform over the ν-subsets of $[m]$ and the nonzero values are uniform on $(\mathbb{Z}_Q \setminus \{0\})^\nu$, so the stated prefix has expected length $H(y) - H(\nu) + O(\log m) \leq H(y) + O(\log m)$, and it implements the round-one action on its own. Strict concavity of h_2 with $h_2(0) = 0$ gives $h_2(\eta u) > \eta h_2(u)$ for $0 < \eta < 1$ and $u = 1 - 1/Q$, which is positivity, since $h_2(u) = h_2(1/Q)$. The derivative in u is $h_2'(\eta u) - h_2'(u) > 0$ because h_2' is decreasing, so the rate increases with Q, to $h_2(\eta)/\eta$ as $u \to 1$. □